

Active Contours & Phân đoạn Cơ bản

Snakes · Energy Minimisation · GVF · GrabCut

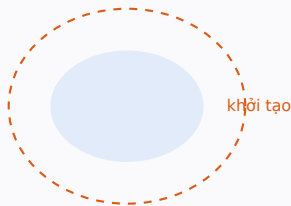
Xử lý ảnh và Thị giác máy tính · 121036

Snakes - Ý tưởng Vật lý (Kass, Witkin & Terzopoulos, 1988)

Hình dung contour như một **dây cao su đàn hồi** đặt trên ảnh.

Dây có hai xu hướng tự nhiên:

1. **Co lại và trơn** (internal forces):
muốn ngắn, muốn cong ít
2. **Bị hút về phía cạnh** (external forces):
vùng gradient lớn = "hố thế năng"



Dây đạt trạng thái cân bằng khi **tổng năng lượng nhỏ nhất**.

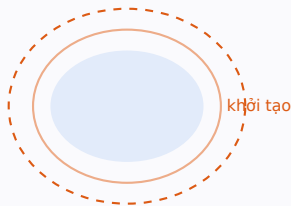
Đây là bài toán **tối ưu hoá**,
không phải bài toán tìm kiếm.

Snakes - Ý tưởng Vật lý (Kass, Witkin & Terzopoulos, 1988)

Hình dung contour như một **dây cao su đàn hồi** đặt trên ảnh.

Dây có hai xu hướng tự nhiên:

1. **Co lại và trơn** (internal forces):
muốn ngắn, muốn cong ít
2. **Bị hút về phía cạnh** (external forces):
vùng gradient lớn = "hố thế năng"



Dây đạt trạng thái cân bằng khi **tổng năng lượng nhỏ nhất**.

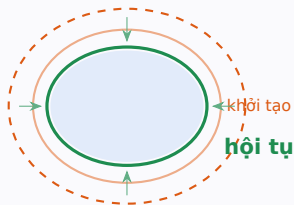
Đây là bài toán **tối ưu hoá**,
không phải bài toán tìm kiếm.

Snakes - Ý tưởng Vật lý (Kass, Witkin & Terzopoulos, 1988)

Hình dung contour như một **dây cao su đàn hồi** đặt trên ảnh.

Dây có hai xu hướng tự nhiên:

1. **Co lại và trơn** (internal forces):
muốn ngắn, muốn cong ít
2. **Bị hút về phía cạnh** (external forces):
vùng gradient lớn = "hố thế năng"



Dây đạt trạng thái cân bằng khi **tổng năng lượng nhỏ nhất**.

Đây là bài toán **tối ưu hoá**,
không phải bài toán tìm kiếm.

Hàm Năng lượng Tổng quát

Contour là tập hợp điểm $\mathbf{v}(s) = (x(s), y(s))$, $s \in [0, 1]$ (tham số cung).

Hàm năng lượng Snakes:

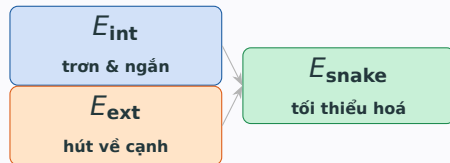
$$E_{\text{snake}} = \int_0^1 \left[\underbrace{E_{\text{int}}(\mathbf{v}(s))}_{\text{internal}} + \underbrace{E_{\text{ext}}(\mathbf{v}(s))}_{\text{external}} \right] ds$$

E_{int} : năng lượng nội tại

- kiểm soát hình dạng của contour
- ưu tiên contour trơn và ngắn

E_{ext} : năng lượng ngoại lực

- đến từ dữ liệu ảnh
- kéo contour về phía cạnh mạnh

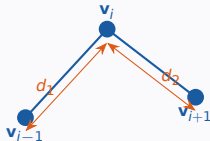


Internal Energy - Continuity Term (α)

$$E_{\text{continuity}} = \alpha \int_0^1 \left| \frac{d\mathbf{v}}{ds} \right|^2 ds \approx \alpha \sum_i |\mathbf{v}_i - \mathbf{v}_{i-1}|^2$$

Ý nghĩa vật lý: đo **tổng độ dài** contour.

- α lớn: contour muốn **co ngắn lại** như dây đàn căng – bỏ qua chi tiết
- α nhỏ: contour được phép dài hơn bám sát mọi chi tiết biên



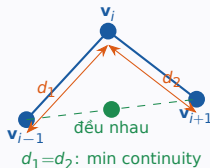
Pitfall: α quá lớn $\Rightarrow$ contour co về một điểm và biến mất.

Internal Energy - Continuity Term (α)

$$E_{\text{continuity}} = \alpha \int_0^1 \left| \frac{d\mathbf{v}}{ds} \right|^2 ds \approx \alpha \sum_i |\mathbf{v}_i - \mathbf{v}_{i-1}|^2$$

Ý nghĩa vật lý: đo **tổng độ dài** contour.

- α lớn: contour muốn **co ngắn lại** như dây đàn căng – bỏ qua chi tiết
- α nhỏ: contour được phép dài hơn bám sát mọi chi tiết biên



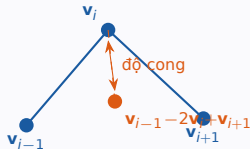
Pitfall: α quá lớn $\Rightarrow$ contour co về một điểm và biến mất.

Internal Energy - Curvature Term (β)

$$E_{\text{curvature}} = \beta \int_0^1 \left| \frac{d^2 \mathbf{v}}{ds^2} \right|^2 ds \approx \beta \sum_i |\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}|^2$$

Ý nghĩa vật lý: đo **độ cong** tại từng điểm.

- β lớn: contour muốn **thẳng hơn** như thanh kim loại cứng - không bẻ góc
- β nhỏ: contour được phép uốn cong bám được góc nhọn, chỗ lồi



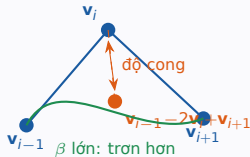
$$E_{\text{int}} = \alpha E_{\text{continuity}} + \beta E_{\text{curvature}}$$

Internal Energy - Curvature Term (β)

$$E_{\text{curvature}} = \beta \int_0^1 \left| \frac{d^2 \mathbf{v}}{ds^2} \right|^2 ds \approx \beta \sum_i |\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}|^2$$

Ý nghĩa vật lý: đo **độ cong** tại từng điểm.

- β lớn: contour muốn **thẳng hơn** như thanh kim loại cứng – không bẻ góc
- β nhỏ: contour được phép uốn cong bám được góc nhọn, chỗ lõm



$$E_{\text{int}} = \alpha E_{\text{continuity}} + \beta E_{\text{curvature}}$$

External Energy - Gradient Magnitude

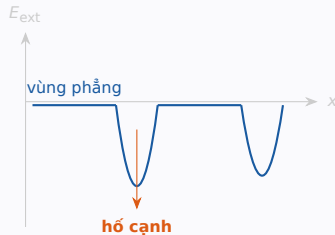
Ý tưởng: cạnh ảnh = nơi gradient lớn = **hố thế năng** hút contour.

$$E_{\text{ext}}(\mathbf{v}) = -|\nabla I(\mathbf{v})|^2$$

- Tại vùng phẳng: $|\nabla I| \approx 0 \Rightarrow E_{\text{ext}} \approx 0$ - không lực kéo
- Tại cạnh mạnh: $|\nabla I|$ lớn $\Rightarrow E_{\text{ext}}$ âm, sâu - hút mạnh

Vấn đề: vùng hút rất hẹp
- chỉ mạnh ngay tại pixel cạnh.

Contour phải khởi tạo **gần vật thể**
để "cảm nhận" được lực kéo.

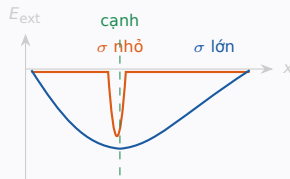


External Energy - Mở rộng Vùng Hút

Vấn đề: $-|\nabla I|^2$ hẹp - contour xa không "cảm thấy" lực.

Giải pháp: $E_{\text{ext}} = -|\nabla(G_{\sigma} * I)|^2$ - làm mờ để mở rộng vùng hút.

- σ lớn: vùng hút rộng - contour "thấy" vật thể từ xa nhưng mất localisation chính xác
- σ nhỏ: vùng hút hẹp - phải khởi tạo rất gần nhưng bám cạnh chính xác hơn



Trade-off: capture range vs. accuracy.

Giải pháp nâng cao: GVF - Phần 3.

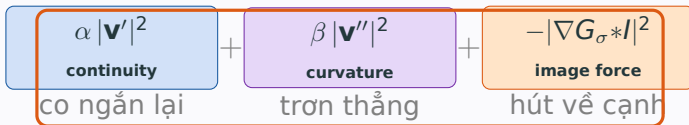
Hàm Năng lượng Đầy đủ - Ba Số hạng

$$E_{\text{snake}} = \int_0^1 \left[\underbrace{\alpha |\mathbf{v}'|^2}_{\text{continuity}} + \underbrace{\beta |\mathbf{v}''|^2}_{\text{curvature}} - \underbrace{|\nabla(G_\sigma * I)|^2}_{\text{image force}} \right] ds$$

$\alpha \mathbf{v}' ^2$ continuity	+	$\beta \mathbf{v}'' ^2$ curvature	+	$- \nabla G_\sigma * I ^2$ image force
co giãn lại		trơn thẳng		hút về cạnh

Hàm Năng lượng Đầy đủ - Ba Số hạng

$$E_{\text{snake}} = \int_0^1 \left[\underbrace{\alpha |\mathbf{v}'|^2}_{\text{continuity}} + \underbrace{\beta |\mathbf{v}''|^2}_{\text{curvature}} - \underbrace{|\nabla(G_\sigma * I)|^2}_{\text{image force}} \right] ds$$



tối thiểu hoá E_{snake} bằng gradient descent

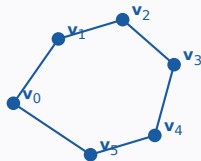
Rời rạc hoá - Từ Tích phân đến Tổng

Trên máy tính, contour là tập hữu hạn n điểm $\{\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{n-1}\}$.

Xấp xỉ đạo hàm rời rạc:

$$\mathbf{v}'_i \approx \mathbf{v}_i - \mathbf{v}_{i-1}$$

$$\mathbf{v}''_i \approx \mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}$$



$n = 6$ điểm rời rạc

Năng lượng rời rạc:

$$E \approx \sum_{i=0}^{n-1} \left[\alpha |\mathbf{v}_i - \mathbf{v}_{i-1}|^2 + \beta |\mathbf{v}_{i-1} - 2\mathbf{v}_i + \mathbf{v}_{i+1}|^2 + E_{\text{ext}}(\mathbf{v}_i) \right]$$

Nhạy cảm với Khởi tạo - Local Minima

Gradient descent chỉ tìm **cực tiểu cục bộ**.

Contour khởi tạo sai $\Rightarrow$ hội tụ về biên sai hoặc không hội tụ.

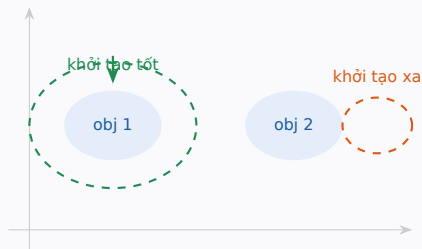


Quy tắc thực tế: khởi tạo contour bao quanh vật thể, cách biên khoảng $r \leq 3\sigma$ **pixel**.

Nhạy cảm với Khởi tạo - Local Minima

Gradient descent chỉ tìm **cực tiểu cục bộ**.

Contour khởi tạo sai $\Rightarrow$ hội tụ về biên sai hoặc không hội tụ.

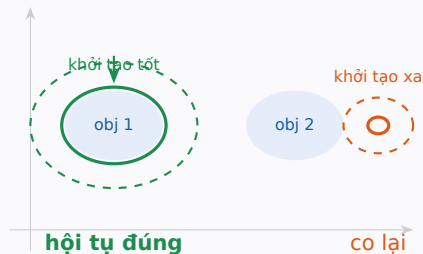


Quy tắc thực tế: khởi tạo contour bao quanh vật thể, cách biên khoảng $r \leq 3\sigma$ **pixel**.

Nhạy cảm với Khởi tạo - Local Minima

Gradient descent chỉ tìm **cực tiểu cục bộ**.

Contour khởi tạo sai $\Rightarrow$ hội tụ về biên sai hoặc không hội tụ.



Quy tắc thực tế: khởi tạo contour bao quanh vật thể, cách biên khoảng $r \leq 3\sigma$ **pixel**.

Ý tưởng cốt lõi - Phần 1

Contour = dây cao su: muốn **ngắn** và **trơn**, đồng thời bị **hút về cạnh**.

Trạng thái cân bằng = cực tiểu của E_{snake} .

$$E_{\text{snake}} = \int_0^1 \underbrace{[\alpha |\mathbf{v}'|^2 + \beta |\mathbf{v}''|^2]}_{E_{\text{int}}} \underbrace{- |\nabla G_{\sigma} * I|^2}_{E_{\text{ext}}} ds$$

tối thiểu hoá bằng gradient descent rời rạc

Trong phần trước...

$$E_{\text{int}} = \alpha |\mathbf{v}'|^2 + \beta |\mathbf{v}''|^2 - \text{trơn và ngăn}$$

$$E_{\text{ext}} = -|\nabla(G_\sigma * I)|^2 - \text{hút về cạnh}$$

Rời rạc hoá: n điểm $\mathbf{v}_i$, đạo hàm sai phân

Phần này: tối ưu hoá, ma trận A , trade-off tham số

Tối ưu hoá - Gradient Descent Rời rạc

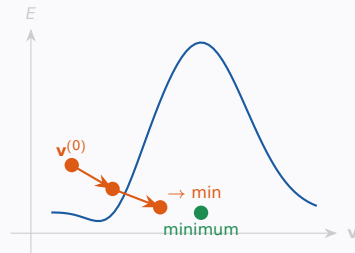
Mỗi điểm $\mathbf{v}_i = (x_i, y_i)$ được cập nhật theo hướng giảm gradient của E_{snake} :

$$\mathbf{v}_i^{(t+1)} = \mathbf{v}_i^{(t)} - \gamma \left. \frac{\partial E_{\text{snake}}}{\partial \mathbf{v}_i} \right|^{(t)}$$

- γ : **bước học** (step size / time step)
- Tất cả n điểm được cập nhật **đồng thời** mỗi iteration
- Lặp đến khi hội tụ
 $\|\mathbf{v}^{(t+1)} - \mathbf{v}^{(t)}\| < \delta$

γ quá lớn $\Rightarrow$ contour dao động, không hội tụ.

γ quá nhỏ $\Rightarrow$ cần rất nhiều iteration.



Ma trận Hệ số A - Implicit Scheme

Gradient của E_{int} theo $\mathbf{v}$ có dạng **tuyến tính**: $\frac{\partial E_{\text{int}}}{\partial \mathbf{v}} = A \mathbf{v}$

A là ma trận **pentadiagonal** (5 đường chéo) kích thước $n \times n$:

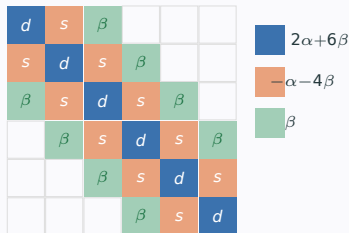
- Đường chéo chính: $2\alpha + 6\beta$
- ± 1 bên: $-\alpha - 4\beta$
- ± 2 bên: β

Cập nhật **implicit** (ổn định hơn explicit):

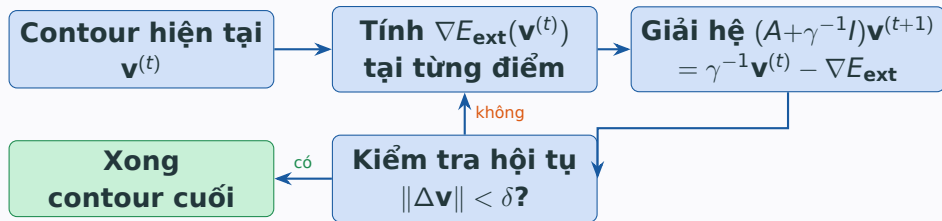
$$(A + \gamma^{-1}I) \mathbf{v}^{(t+1)} = \gamma^{-1} \mathbf{v}^{(t)} - \nabla E_{\text{ext}}$$

Giải hệ tuyến tính mỗi bước - $O(n)$ do A là banded.

A (pentadiagonal)



Một Iteration Hoàn chỉnh



$\nabla E_{\text{ext}}(\mathbf{v}_i) = -2 \nabla \left[|\nabla(G_\sigma * I)|^2 \right]_{\mathbf{v}_i}$ - nội suy bằng bilinear interpolation tại vị trí contour.

Tham số α - Continuity Weight

α nhỏ: bám sát

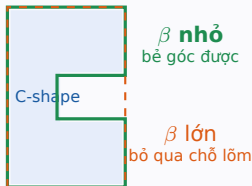


α lớn: co, bỏ chi tiết

α	Hiệu ứng
≈ 0.01	Bám sát biên; nhạy cảm với nhiễu cục bộ
≈ 0.1	Cân bằng; hầu hết các ứng dụng
≈ 1.0	Contour mượt; bỏ qua chi tiết biên nhỏ; có thể co biến mất

Quy tắc thực tế: bắt đầu với $\alpha = 0.01$; tăng nếu contour dao động hoặc bị nhiễu kéo.

Tham số β - Curvature Weight



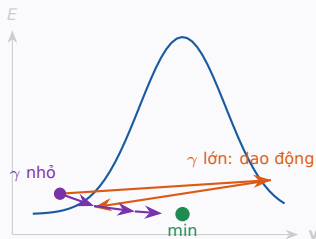
β	Hiệu ứng	Quy
≈ 0.0	Cho phép góc nhọn; dễ bị nhiễu cục bộ	
≈ 0.1	Trơn vừa; chuẩn cho hầu hết ảnh y tế	
≈ 1.0	Contour rất cứng; không thể vào chỗ lõm	

tắc thực tế: $\beta = 0.1$ là điểm khởi đầu tốt; giảm nếu cần bám biên góc nhọn hoặc chỗ lõm.

Tham số γ - Step Size

γ kiểm soát tốc độ hội tụ và ổn định số học.

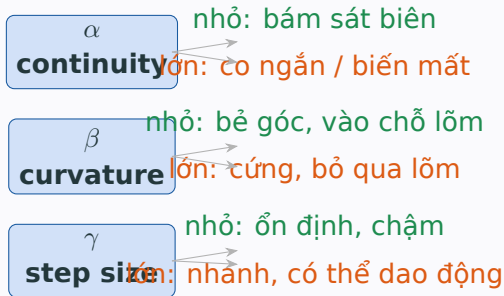
- γ lớn: ít iteration hơn, nhưng có thể **dao động** qua cực tiểu
- γ nhỏ: ổn định, chính xác, nhưng **chậm** - cần nhiều iteration
- Scheme implicit cho phép γ lớn hơn
scheme explicit mà vẫn ổn định



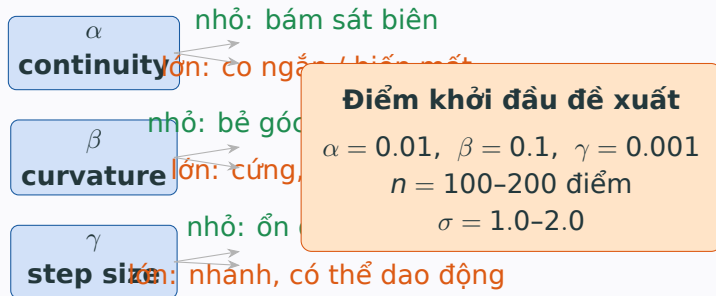
Trong `skimage.active_contour`:

tham số `max_num_iter` (số vòng lặp) và `convergence` (ngưỡng dừng) thay cho γ trực tiếp.

Ba Tham số - Tổng hợp Trade-off



Ba Tham số - Tổng hợp Trade-off



Vấn đề: Concavity Problem

Snakes cố định **không thể** lọt vào vùng lõm sâu.

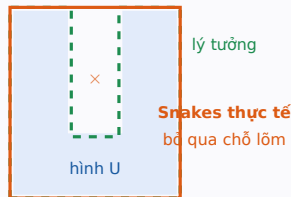
Tại sao?

Lực E_{ext} tại vùng lõm nhỏ hoặc hướng ra ngoài.

Lực E_{int} (continuity) kéo contour **thẳng qua** chỗ lõm.

Biểu hiện:

- Biên chữ U, chữ C, hình sao
- Contour "cắt qua" chỗ lõm thay vì đi vào
- Tăng β làm vấn đề trầm trọng hơn

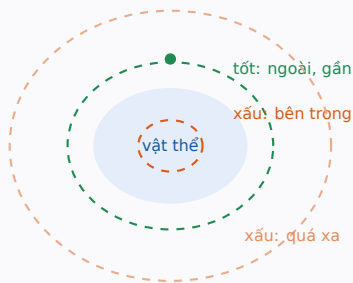


Giải pháp: GVF – sẽ nói ở Phần 3.

Vấn đề: Nhạy với Khởi tạo - Chi tiết

Ba quy tắc thực tế để khởi tạo contour hiệu quả:

1. **Bao quanh** vật thể, không cắt vào trong.
Contour bên trong $\rightarrow$ lực E_{ext} có thể đẩy ra ngoài thay vì co vào.
2. Khoảng cách đến biên $\leq 3\sigma$
Ngoài capture range của $E_{\text{ext}} \rightarrow$ contour co lại không tìm được cạnh.
3. Số điểm n đủ dày
Quá ít điểm $\rightarrow$ đoạn thẳng dài giữa hai điểm; curvature term thống trị.



API chính:

- `active_contour(image, snake, ...)`
- `image`: ảnh float, đã Gaussian blur
- `snake`: array ($n, 2$) các điểm (y, x) khởi tạo
- `alpha, beta, gamma`: 3 tham số chính
- `max_num_iter`: số vòng lặp tối đa
- `convergence`: ngưỡng dừng sớm

Output: array ($n, 2$) – vị trí cuối của contour

```
from skimage.filters import gauss
from skimage.segmentation \
    import active_contour
```

```
img_blur = gaussian(img, sigma=2)
```

```
# Khởi tạo hình tròn
t = np.linspace(0, 2*np.pi, 200)
init = np.c_[
    cy + r*np.sin(t),
    cx + r*np.cos(t)
]
```

```
snake = active_contour(
    img_blur, init,
```

Demo 2 - active_contour với skimage

Demo 2: Quan sát ảnh hưởng trực tiếp của α và β

1. `skimage.data.coins()` - crop một đồng xu
2. Khởi tạo contour tròn bên ngoài
3. Chạy 3 bộ (α, β) khác nhau
4. Plot 3 kết quả cạnh nhau
5. Giải thích tại sao mỗi bộ cho kết quả khác

Bộ tham số: (0.01, 0.1) - tight fit | (0.1, 0.1) - over-smooth | (0.01, 1.0) - stiff

Ý tưởng cốt lõi - Phần 2

A pentadiagonal + implicit scheme $\Rightarrow$ giải $O(n)$ mỗi iteration.

α : co/dài – β : cứng/mềm – γ : nhanh/ổn định.

Hai giới hạn chính: nhạy với khởi tạo & không vào được chỗ lõm.

$$(A + \gamma^{-1}I) \mathbf{v}^{(t+1)} = \gamma^{-1}\mathbf{v}^{(t)} - \nabla E_{\text{ext}}$$

Phần 3: GVF giải concavity, GrabCut cầu nối sang region-based

Trong phần trước...

$$(A + \gamma^{-1}I)\mathbf{v}^{(t+1)} = \gamma^{-1}\mathbf{v}^{(t)} - \nabla E_{\text{ext}}$$

α : co/dài – β : cứng/mềm – γ : nhanh/ổn định

Hai giới hạn: nhạy khởi tạo & không vào được chỗ lõm

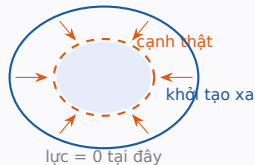
Phần này: GVF, Balloon, GrabCut & tổng kết

Nhắc lại - Hai Giới hạn Cần Giải Quyết

Giới hạn 1: Capture range hẹp

$E_{\text{ext}} = -|\nabla(G_{\sigma} * I)|^2$ chỉ mạnh **ngay tại cạnh**.

Contour khởi tạo xa $\Rightarrow$ không "cảm thấy" lực.

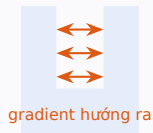


Giới hạn 2: Concavity problem

Gradient thông thường hướng **ra ngoài** tại chỗ lõm.

Contour không thể bị hút vào.

contour không vào được



GVF - Gradient Vector Flow (Xu & Prince, 1998)

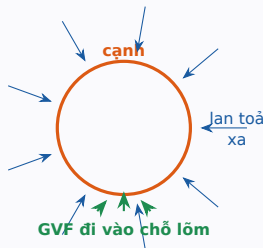
Ý tưởng: thay vì dùng gradient ảnh trực tiếp, **lan toả** (diffuse) trường vector lực ra khắp ảnh.

$$\mathbf{f}(x, y) = (u(x, y), v(x, y))$$

Tính GVF bằng cách tối thiểu hoá:

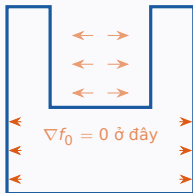
$$\iint \mu(|\nabla u|^2 + |\nabla v|^2) + |\nabla f_0|^2 |\mathbf{f} - \nabla f_0|^2 dA$$

- $f_0 = |\nabla(G_\sigma * I)|$: edge map gốc
- μ : hệ số khuếch tán - kiểm soát độ lan toả
- Tại cạnh: $\mathbf{f} \approx \nabla f_0$ (giữ nguyên)

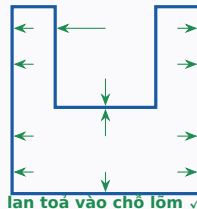


GVF - Gradient Thường vs. GVF Field

Gradient thường ∇f_0



GVF field f



Balloon Force - Giải Capture Range

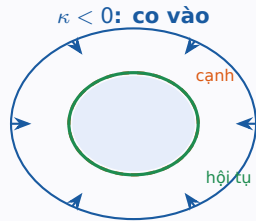
Balloon Snake (Cohen, 1991): thêm lực **áp suất** giúp contour chủ động phình to hoặc co lại.

$$E_{\text{balloon}} = -\kappa \int_0^1 \mathbf{n}(s) \cdot \mathbf{v}(s) ds$$

- $\mathbf{n}(s)$: véc-tơ pháp tuyến ngoài tại s
- $\kappa > 0$: contour **phình to** – tiến về phía cạnh từ trong ra
- $\kappa < 0$: contour **co lại** – tiến từ ngoài vào trong

Tác dụng: contour không cần khởi tạo gần vật thể; lực áp suất đưa nó vào vùng hút của E_{ext} .

Pitfall: nếu $|\kappa|$ quá lớn, contour vượt qua cạnh.



So sánh: Snakes Cổ điển / GVF / Balloon

Tiêu chí	Snakes cổ điển	GVF Snake	Balloon Snake
Capture range	hẹp	rộng	rộng (nhờ κ)
Concavity	không vào được	vào được	tuỳ β
Tham số thêm	-	μ (khuếch tán)	κ (áp suất)
Chi phí tính toán	thấp	cao (giải PDE)	thấp
Nhạy với khởi tạo	cao	trung bình	thấp

Trong thực tế: **GVF Snake** là lựa chọn mạnh nhất trong họ Snakes; tuy nhiên cả họ này đều bị thay thế dần bởi phương pháp **level set** và **deep segmentation** cho ứng dụng phức tạp.

Từ Contour-based đến Region-based

Snakes (contour-based):

- Tìm **đường biên** trực tiếp
- Phụ thuộc gradient cạnh
- Khởi tạo thủ công
- Tốt với biên sắc nét, ít nhiễu

Điểm yếu:

Cạnh yếu, mờ, hoặc bị che khuất
⇒ contour bị kéo sai hoặc rò rỉ.

Region-based:

- Phân biệt **vùng màu/kết cấu**
- Không cần cạnh rõ nét
- Biên suy ra từ ranh giới vùng
- Robust hơn với nhiễu và che khuất

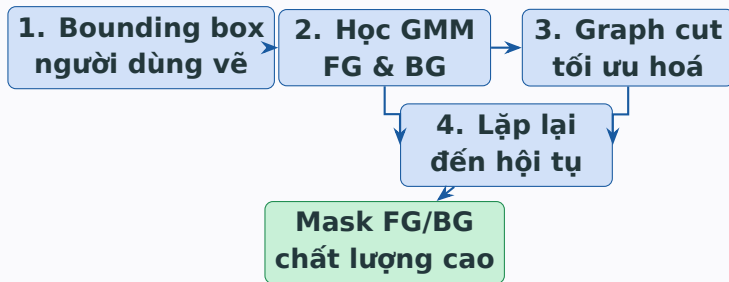
GrabCut

Mean-Shift

Chan-Vese / Level Sets

GrabCut - Region-based Segmentation (Rother et al., 2004)

GrabCut kết hợp mô hình màu (GMM) và **graph cut** tối ưu hoá.



GMM (Gaussian Mixture Model): mô hình hoá màu FG và BG bằng $K = 5$ Gaussian.

Graph cut: mỗi pixel là node; cut tối thiểu = phân vùng tốt nhất.

GrabCut vs. Snakes - Hai Triết lý

Snakes:

Tối ưu hoá đường cong $\mathbf{v}(s)$

Minimise $E = E_{\text{int}} + E_{\text{ext}}$

Kết quả: tập điểm trên biên

điểm contour



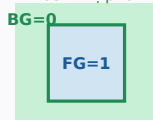
GrabCut:

Tối ưu hoá nhãn pixel $L_i \in \{0, 1\}$

Minimise energy trên graph

Kết quả: mặt nạ nhị phân

mask nhị phân



Cả hai đều là bài toán tối ưu hoá – khác nhau ở đối tượng tối ưu: đường cong liên tục vs. nhãn rời rạc.

- **Kass, Witkin & Terzopoulos (1988)** – “Snakes: Active Contour Models.” *IJCV* 1(4).
Bài gốc; dễ đọc; Mục 2-3 là cốt lõi.
- **Xu & Prince (1998)** – “Snakes, Shapes, and Gradient Vector Flow.” *IEEE TIP* 7(3).
GVF đầy đủ; xem Figure 2-4 để hiểu trực quan trường vector.
- **Rother, Kolmogorov & Blake (2004)** – “GrabCut.” *ACM SIGGRAPH*.
Giải thích GMM + graph cut rõ ràng; ngắn gọn (8 trang).
- **Szeliski (2022)** – *Computer Vision: Algorithms and Applications*, Chương 5.3-5.4.
Bao phủ Snakes, GVF, level sets và GrabCut trong một mạch.

Active Contours & Phân đoạn Cơ bản

Tuần 9 – Xử lý ảnh và Thị giác máy tính

Canny → Hough / RANSAC → Snakes / GrabCut
pixel → primitive → **region**

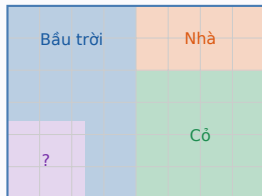
Các tuần tiếp theo: Deep Learning Segmentation – U-Net, SAM

TUẦN 10 · XỬ LÝ ẢNH VÀ THỊ GIÁC MÁY TÍNH

Phân Đoạn Vùng

Region-Based Image Segmentation

Region Growing · Split-and-Merge · Watershed



Những pixel nào thuộc về cùng một vùng?

- Tuần 7-9: tìm **cạnh**, **đường thẳng**, **contour**
→ xác định *biên* giữa các vùng
- Tuần 10: tiếp cận ngược lại —
→ xác định trực tiếp *vùng* bên trong

Câu hỏi cốt lõi

Cho một pixel bất kỳ, làm sao biết nó "giống" các pixel lân cận đến đâu?

Ba chiến lược phân đoạn vùng



Tất cả đều trả lời: "Pixel này thuộc về vùng nào?"

Hôm nay — Phần 1

Seed → lan ra theo tiêu chí đồng nhất

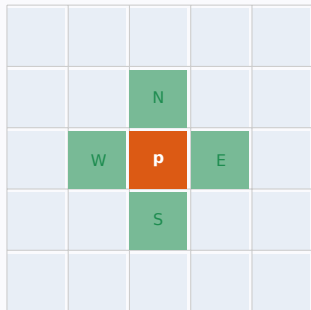
Hôm nay — Phần 2

Chia nhỏ nếu không đồng nhất, hợp nếu tương đồng

Hôm nay — Phần 3

Gradient = địa hình; flood từ cực tiểu

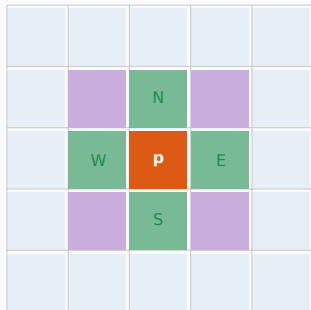
Nhắc lại: kết nối pixel



Kết nối 4 (4-connectivity)

N, S, E, W — 4 pixel lân cận trực tiếp

Nhắc lại: kết nối pixel



Kết nối 4 (4-connectivity)

N, S, E, W — 4 pixel lân cận trực tiếp

Kết nối 8 (8-connectivity)

Thêm 4 pixel ở góc chéo

Lưu ý thực tế

Kết nối 8 bao phủ rộng hơn, nhưng có thể tạo ra "đường chéo" không mong muốn.

Chọn kết nối *trước* khi chạy thuật toán.

Tiêu chí đồng nhất (Homogeneity Predicate)

Để quyết định có "chấp nhận" pixel q vào vùng hay không, cần một hàm $P(R)$ trả về true/false:

Định nghĩa tổng quát

$P(R) = \text{true}$ nếu vùng R **đồng nhất** theo tiêu chí đã chọn.

Ví dụ thường dùng:

- **Ngưỡng cường độ:**

$$|I(q) - \mu_R| < \tau$$

- **Phương sai:** $\sigma_R^2 < \tau$

- **Khoảng cách màu:**

$$\|C(q) - \bar{C}_R\| < \tau$$

Tham số quan trọng:

- μ_R : cường độ trung bình của vùng R
- σ_R^2 : phương sai trong vùng R
- τ : **ngưỡng** — người dùng đặt

Câu hỏi thiết kế

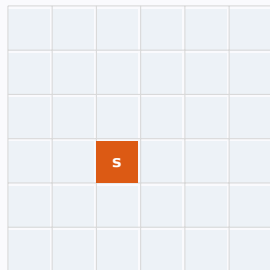
τ lớn $\rightarrow$ vùng lớn, đồng nhất kém. τ nhỏ $\rightarrow$ vùng nhỏ, quá phân mảnh.

Không có τ "đúng" duy nhất — phụ thuộc vào ảnh và mục đích.

Thuật toán Region Growing

Đầu vào: ảnh I , seed pixel s , tiêu chí P , kết nối k

1. Khởi tạo: $R \leftarrow \{s\}$, hàng đợi $Q \leftarrow \{s\}$
2. Lấy p từ Q
3. Với mỗi pixel $q \in \mathcal{N}_k(p)$:
 - Nếu $q \notin R$ và $P(R \cup \{q\}) = \text{true}$
→ thêm q vào R và Q
4. Lặp lại 2-3 cho đến khi Q rỗng
5. Trả về vùng R

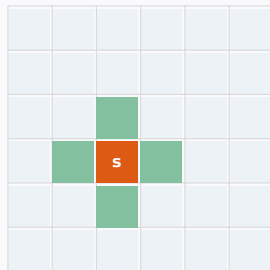


Bản chất: **BFS** (Breadth-First Search) có điều kiện — hàng đợi Q chính là queue của BFS.

Thuật toán Region Growing

Đầu vào: ảnh I , seed pixel s , tiêu chí P , kết nối k

1. Khởi tạo: $R \leftarrow \{s\}$, hàng đợi $Q \leftarrow \{s\}$
2. Lấy p từ Q
3. Với mỗi pixel $q \in \mathcal{N}_k(p)$:
 - Nếu $q \notin R$ và $P(R \cup \{q\}) = \text{true}$
→ thêm q vào R và Q
4. Lặp lại 2-3 cho đến khi Q rỗng
5. Trả về vùng R

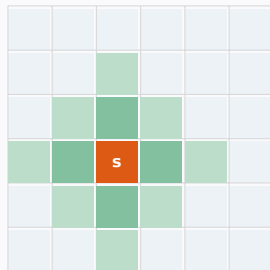


Bản chất: **BFS** (Breadth-First Search) có điều kiện — hàng đợi Q chính là queue của BFS.

Thuật toán Region Growing

Đầu vào: ảnh I , seed pixel s , tiêu chí P , kết nối k

1. Khởi tạo: $R \leftarrow \{s\}$, hàng đợi $Q \leftarrow \{s\}$
2. Lấy p từ Q
3. Với mỗi pixel $q \in \mathcal{N}_k(p)$:
 - Nếu $q \notin R$ và $P(R \cup \{q\}) = \text{true}$
→ thêm q vào R và Q
4. Lặp lại 2-3 cho đến khi Q rỗng
5. Trả về vùng R

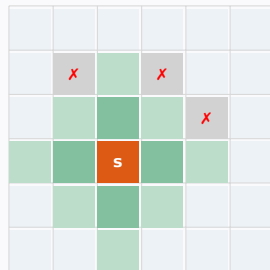


Bản chất: **BFS** (Breadth-First Search) có điều kiện — hàng đợi Q chính là queue của BFS.

Thuật toán Region Growing

Đầu vào: ảnh I , seed pixel s , tiêu chí P , kết nối k

1. Khởi tạo: $R \leftarrow \{s\}$, hàng đợi $Q \leftarrow \{s\}$
2. Lấy p từ Q
3. Với mỗi pixel $q \in \mathcal{N}_k(p)$:
 - Nếu $q \notin R$ và $P(R \cup \{q\}) = \text{true}$
→ thêm q vào R và Q
4. Lặp lại 2-3 cho đến khi Q rỗng
5. Trả về vùng R



Bản chất: **BFS** (Breadth-First Search) có điều kiện — hàng đợi Q chính là queue của BFS.

Minh hoạ: Region Growing từng bước

50 46 44 105 110 118

47	44	42	108	113	120
48	45	110	112	116	122
46	43	39	115	119	125
44	40	41	118	122	128
40	42	38	120	125	130

S

Minh hoạ: Region Growing từng bước

50 46 44 105 110 118

47	44	42	108	113	120
48	45	110	112	116	122
46	43	39	115	119	125
44	40	41	118	122	128
40	42	38	120	125	130

Vòng 1: $|v-43|<10$

Minh hoạ: Region Growing từng bước

50 46 44 105 110 118

47	44	42	108	113	120
48	45	110	112	116	122
46	43	39	115	119	125
44	40	41	118	122	128
40	42	38	120	125	130

Vòng 1: $|v-43|<10$

Vòng 2: lan tiếp

Minh hoạ: Region Growing từng bước

50 46 44 105 110 118

47	44	42	108	113	120
48	45	110	112	116	122
46	43	39	115	119	125
44	40	41	118	122	128
40	42	38	120	125	130

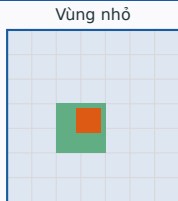
Vòng 1: $|v-43|<10$

Vòng 2: lan tiếp

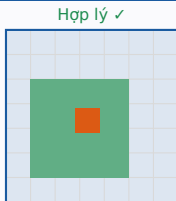
(2,3)=110: $|110-43|=67 \rightarrow$ từ chối

Ranh giới tự nhiên!

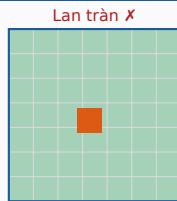
Ngưỡng τ quyết định kích thước vùng



$\tau = 5$



$\tau = 15$



$\tau = 50$

Nhận xét

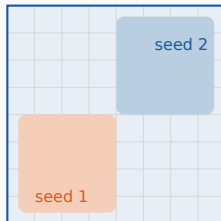
- τ **quá nhỏ**: vùng không phát triển đủ — phân mảnh quá mức
- τ **quá lớn**: vùng lan sang cả vùng khác — mất ranh giới
- Không có τ "đúng" toàn cục — thường dùng **adaptive thresholding** hoặc điều chỉnh tương tác

Vấn đề chọn Seed

Region Growing phụ thuộc mạnh vào **vị trí seed** và **số lượng seed**.

Cách chọn seed thông dụng:

1. **Thủ công**: người dùng click vào điểm quan tâm
2. **Cực trị cục bộ**: pixel có cường độ cực đại/cực tiểu
3. **Lưới đều**: đặt seed theo lưới đều trên ảnh
4. **Từ kết quả thuật toán khác**: ví dụ dùng điểm góc Harris làm seed



Hạn chế của Region Growing

Kết quả *không duy nhất*: thay đổi seed → thay đổi vùng. Không có cơ

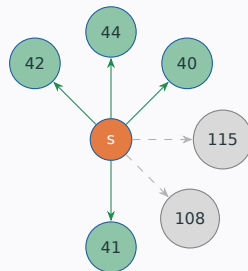
Region Growing = BFS có điều kiện

BFS chuẩn (duyệt đồ thị):

- Khởi đầu từ node gốc
- Thêm *mọi* hàng xóm chưa thăm vào hàng đợi
- Duyệt hết tất cả node liên thông

Region Growing (duyệt pixel):

- Khởi đầu từ seed pixel
- Chỉ thêm hàng xóm *thỏa* $P(R)$
- Duyệt đến khi không còn pixel thỏa điều kiện



115, 108: $|v-43| > 10 \rightarrow \times$

Kết luận

Nếu đã biết BFS, bạn đã hiểu 80% thuật toán Region Growing.

Điểm mới duy nhất là **tiêu chí đồng nhất P** .

Độ phức tạp và giới hạn

Độ phức tạp thời gian:

$$O(N \cdot k)$$

- N : số pixel trong ảnh
- k : số hàng xóm mỗi pixel (4 hoặc 8)
- Mỗi pixel được thêm/kiểm tra tối đa một lần
- Hiệu quả — tương đương BFS tiêu chuẩn

Giới hạn thực tế:

- **Nhạy cảm với seed**: thay seed → thay kết quả
- **Nhạy cảm với τ** : không tự chọn ngưỡng
- **Nhiều**: pixel nhiễu đơn lẻ có thể chặn lan rộng
- **Không tìm được nhiều vùng**: phải chạy nhiều lần với nhiều seed

Thích hợp nhất khi có **seed xác định sẵn** và **ngưỡng biết trước**

Ý tưởng cốt lõi — Phần 1

Region Growing = **BFS** + **tiêu chí đồng nhất P**

Chấp nhận $q \iff P(R \cup \{q\}) = \text{true}$

Seed xác định
vùng bắt đầu

τ xác định
biên vùng

Kết quả phụ thuộc
cả hai tham số

Tiếp theo: Split-and-Merge — không cần seed, nhưng cần tiêu chí chia đôi $\rightarrow$

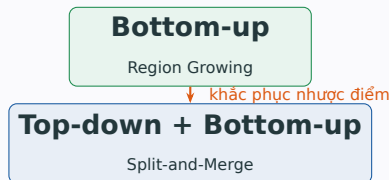
Trong phần trước — Region Growing

Đã học: Region Growing = BFS +
tiêu chí P

- Seed $\rightarrow$ lan ra theo $|I(q) - \mu_R| < \tau$
- Kết quả phụ thuộc vào **seed** và τ
- Không tự tìm được tất cả các vùng

Câu hỏi còn mở:

Nếu không có seed, làm sao bắt đầu?



Ý tưởng: Chia rồi Hợp

Chiến lược hai pha:

1. **Split** (Chia): Bắt đầu từ *toàn bộ ảnh*.

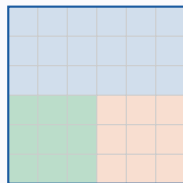
Nếu vùng **không đồng nhất** → chia thành 4 ô con.

Lặp lại đệ quy cho đến khi mọi vùng đồng nhất.

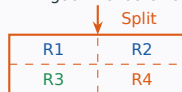
2. **Merge** (Hợp): Duyệt các vùng lân cận.

Nếu hai vùng **tương đồng** → hợp nhất thành một.

Lặp lại cho đến khi không còn cặp nào có thể hợp.

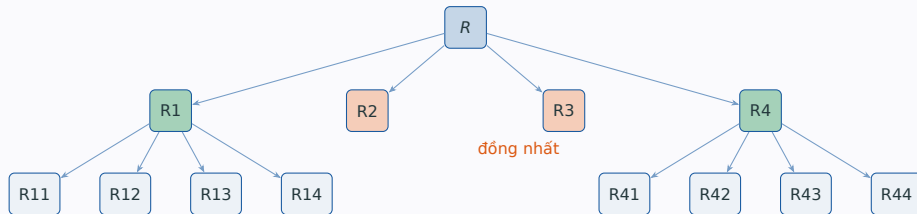


Ảnh gốc — chưa chia



Sau Split

Cấu trúc dữ liệu: Quad-tree



đồng nhất

Mỗi node trong **quad-tree** biểu diễn một **vùng chữ nhật** của ảnh:

- **Node gốc** = toàn bộ ảnh
- Mỗi node có thể chia thành **4 con** (NW, NE, SW, SE)
- **Node lá** = vùng đồng nhất, không chia thêm

Ràng buộc quan trọng

Ảnh phải có kích thước $2^n \times 2^n$ (hoặc padding về kích thước này).

Pha Split: Chia đệ quy

$P(R) = \text{false}$ — chia!

42	44	125	128
40	46	122	130
43	41	118	126
45	39	120	124

Bước 0: Ảnh gốc

Pha Split: Chia đệ quy

$P(R) = \text{false}$ — chia!

42	44	125	128
40	46	122	130
43	41	118	126
45	39	120	124

Bước 0: Ảnh gốc

42	44	125	128
SW ✓		SE ✗	
40	46	122	130
43	41	118	126
NW ✓		NE ✗	
45	39	120	124

Bước 1: Chia 4

Pha Split: Chia đệ quy

$P(R) = \text{false}$ — chia!

42	44	125	128
40	46	122	130
43	41	118	126
45	39	120	124

Bước 0: Ảnh gốc

42	44	125	128
40	46	122	130
43	41	118	126
45	39	120	124

Bước 1: Chia 4

Lá đồng nhất ✓

42	44	125	128
40	46	122	130
43	41	118	126
45	39	120	124

Bước 2: Chia tiếp

Điều kiện Split: khi nào chia?

Chia vùng R khi $P(R) = \text{false}$, tức là vùng **không đồng nhất**.

Tiêu chí phương sai (thường dùng nhất):

$$P(R) = \begin{cases} \text{true} & \text{nếu } \sigma^2(R) \leq \tau \\ \text{false} & \text{nếu } \sigma^2(R) > \tau \end{cases}$$

$$\text{Với } \sigma^2(R) = \frac{1}{|R|} \sum_{p \in R} (I(p) - \mu_R)^2$$

Ví dụ: $\tau = 100$

$$\mu = 130, \sigma^2 = 18 \checkmark$$

Giữ nguyên

$$\mu = 85, \sigma^2 = 420 \times$$

Chia tiếp!

Lưu ý tính nhất quán

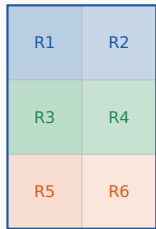
Tiêu chí Split và Merge *phải* tương thích nhau.

Nếu không, một vùng có thể bị chia rồi lại hợp liên tục — **không hội tụ**.

Pha Merge: Hợp các vùng lân cận

Sau Split, duyệt tất cả cặp vùng **lân cận** R_i, R_j :

$$\text{Hợp } R_i \cup R_j \iff P(R_i \cup R_j) = \text{true}$$



Sau Split

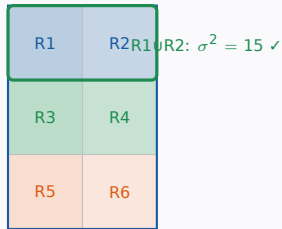
Pha Merge: Hợp các vùng lân cận

Sau Split, duyệt tất cả cặp vùng **lân cận** R_i, R_j :

$$\text{Hợp } R_i \cup R_j \iff P(R_i \cup R_j) = \text{true}$$

Điều kiện "lân cận":

- Hai vùng *chia sẻ ít nhất một cạnh chung*
- Không yêu cầu cùng kích thước



Sau Split

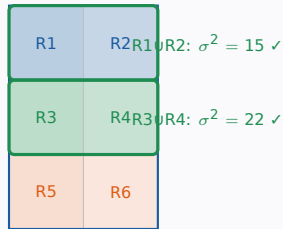
Pha Merge: Hợp các vùng lân cận

Sau Split, duyệt tất cả cặp vùng **lân cận** R_i, R_j :

$$\text{Hợp } R_i \cup R_j \iff P(R_i \cup R_j) = \text{true}$$

Điều kiện "lân cận":

- Hai vùng *chia sẻ ít nhất một cạnh chung*
- Không yêu cầu cùng kích thước



Sau Split

Lặp lại merge cho đến khi không còn cặp nào hợp được.

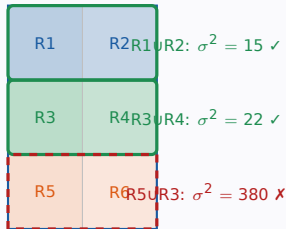
Pha Merge: Hợp các vùng lân cận

Sau Split, duyệt tất cả cặp vùng **lân cận** R_i, R_j :

$$\text{Hợp } R_i \cup R_j \iff P(R_i \cup R_j) = \text{true}$$

Điều kiện "lân cận":

- Hai vùng *chia sẻ ít nhất một cạnh chung*
- Không yêu cầu cùng kích thước



Sau Split

Lặp lại merge cho đến khi không còn cặp nào hợp được.

Kết quả cuối

3 vùng: Top (xanh), Middle (lục), Bottom (cam) — phản ánh đúng cấu trúc ảnh.

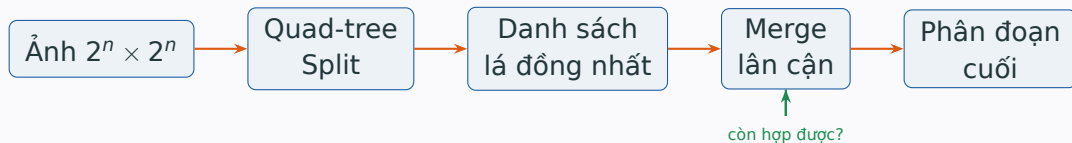
Thuật toán Split-and-Merge đầy đủ

Pha 1 — Split (đệ quy):

1. $R_0 \leftarrow$ toàn bộ ảnh
2. Với mỗi vùng R :
 - Nếu $P(R) = \text{false} \rightarrow$ chia thành $R_{NW}, R_{NE}, R_{SW}, R_{SE}$
 - Đệ quy cho 4 ô con
3. Dừng khi mọi lá đều đồng nhất hoặc đạt kích thước tối thiểu (vd: 1×1 pixel)

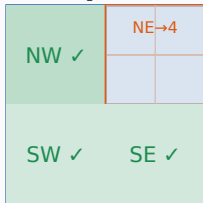
Pha 2 — Merge (lặp):

1. Xây danh sách tất cả cặp (R_i, R_j) lân cận
2. Nếu $P(R_i \cup R_j) = \text{true} \rightarrow$ hợp nhất: $R_i \leftarrow R_i \cup R_j$, xóa R_j
3. Cập nhật danh sách láng giềng
4. Lặp cho đến khi không còn hợp được

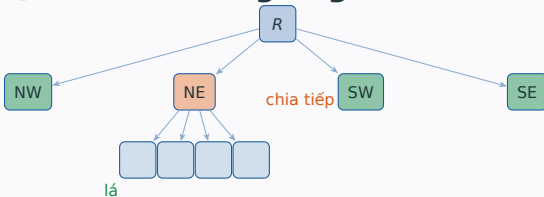


Quad-tree: cây biểu diễn và lưới phân chia

Lưới phân chia (image space):



Quad-tree tương ứng:



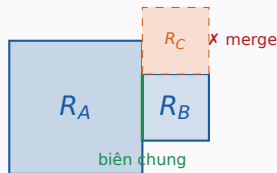
Node lá **xanh** = vùng đồng nhất.

Node **cam** = tiếp tục chia.

Tình tế: Merge giữa vùng không cùng kích thước

Sau Split, có thể xảy ra tình huống:

- Vùng lớn R_A (2×2) lân cận với vùng nhỏ R_B (1×1)
- Cả hai đồng nhất nội bộ
- $P(R_A \cup R_B) = \text{true} \rightarrow$ có thể merge



Điều kiện "lân cận" tổng quát
 R_i và R_j lân cận nếu chúng chia sẻ ít nhất một đoạn biên chung — *bất kể kích thước*.

Tại sao điều này quan trọng?

Nếu chỉ merge vùng cùng cấp $\rightarrow$ bỏ sót nhiều hợp nhất hợp lệ.

Implementation cần kiểm tra *tất cả* cặp lân cận, không chỉ cùng cha.

So sánh: Split-and-Merge vs. Region Growing

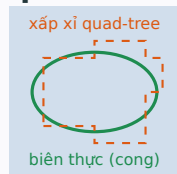
Tiêu chí	Region Growing	Split-and-Merge
Cần seed?	Có	Không
Hướng xử lý	Bottom-up	Top-down + Bottom-up
Số vùng đầu ra	1 vùng / seed	Tự động
Nhạy với tham số	τ , seed	Chỉ τ
Cấu trúc dữ liệu	Queue (BFS)	Quad-tree
Phức tạp thời gian	$O(N)$	$O(N \log N)$
Kết quả ổn định?	Phụ thuộc seed	Ổn định hơn

Khi nào dùng Split-and-Merge?

- Khi *không biết trước* vị trí của các vùng quan tâm
- Khi cần phân đoạn *toàn bộ ảnh* thành nhiều vùng đồng nhất
- Khi ảnh có cấu trúc phân cấp rõ ràng (vùng lớn chứa vùng nhỏ)

Hạn chế của Split-and-Merge

1. Biên vuông — artefact quad-tree:



2. Chi phí merge cao:

- Sau Split có thể có $O(N)$ vùng nhỏ
- Kiểm tra tất cả cặp lân cận tốn $O(N^2)$ trong trường hợp xấu
- Cần cấu trúc dữ liệu thông minh (graph, union-find)

3. Ảnh phải là $2^n \times 2^n$:

- Cần padding trước khi xử lý
- Padding có thể tạo ra vùng giả ở biên ảnh

Ý tưởng cốt lõi — Phần 2

Split-and-Merge = **Quad-tree** + tiêu chí đồng nhất P nhất quán

Split: $P(R) = \text{false} \Rightarrow$ chia 4

Merge: $P(R_i \cup R_j) = \text{true} \Rightarrow$ hợp



Merge (bottom-up)

Không cần seed · Tự động tìm tất cả vùng · Nhưng biên có dạng bậc thang

Tiếp theo: Watershed — dùng gradient như địa hình để tìm biên chính xác hơn



Trong phần trước — Split-and-Merge

Đã học: Quad-tree Split-and-Merge

- Không cần seed — bắt đầu từ toàn ảnh
- Split nếu $P(R) = \text{false}$, Merge nếu $P(R_i \cup R_j) = \text{true}$
- Tự động tìm tất cả các vùng

Hạn chế còn lại:

- Biên có dạng **bậc thang** (artefact quad-tree)
- Không tận dụng thông tin **gradient** tại biên thực

Split-and-Merge

biên bậc thang

▼ khắc phục

Watershed

biên dọc theo gradient

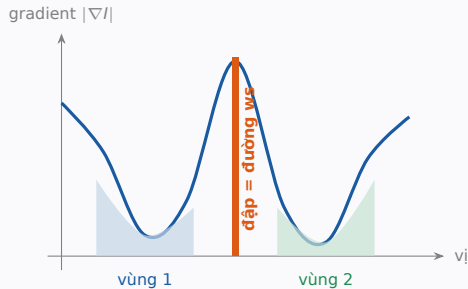
Phép ẩn dụ: Gradient = Địa hình

Gradient magnitude của ảnh trông giống một bản đồ địa hình:

- Vùng phẳng (cường độ đồng đều) → **thung lũng**
gradient thấp
- Biên vật thể (cường độ thay đổi mạnh) → **núi/sườn**
gradient cao

Ý tưởng Watershed:

Hãy tưởng tượng cho **nước ngập** dâng lên từ từ từ các thung lũng. Khi nước từ hai thung lũng khác nhau sắp gặp nhau, ta xây một **đập ngăn** — đó chính là đường phân đoạn.



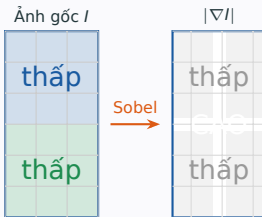
Từ profil 1D sang gradient 2D

Ảnh xám → Gradient magnitude:

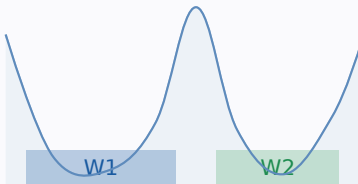
$$|\nabla I(x, y)| = \sqrt{G_x^2 + G_y^2}$$

- Tính bằng bộ lọc Sobel
- Giá trị cao = biên đối tượng
- Giá trị thấp = vùng đồng nhất

Watershed hoạt động trực tiếp trên **ảnh gradient magnitude** — không phải ảnh gốc.

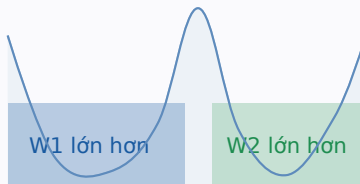


Thuật toán Watershed: Nước dâng từng bước



Mức nước $h = 0.5$: hai hồ độc lập

Thuật toán Watershed: Nước dâng từng bước



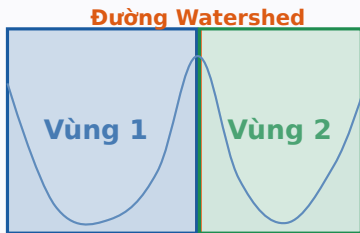
Mức nước $h = 1.2$: hai hồ vẫn tách nhau

Thuật toán Watershed: Nước dâng từng bước



$h = 2.6$: nước sắp gặp nhau $\rightarrow$ xây đập

Thuật toán Watershed: Nước dâng từng bước



Kết quả: 2 vùng tách bởi đường watershed

Thuật toán Watershed rời rạc

Trong thực tế, "nước dâng" được mô phỏng bằng **priority queue**:

1. Tính $|\nabla I|$ cho toàn ảnh
2. Tìm tất cả **cực tiểu cục bộ** (local minima)
→ khởi tạo mỗi cực tiểu là một nhãn vùng
3. Đưa tất cả pixel lân cận của cực tiểu vào **priority queue** (sắp xếp theo $|\nabla I|$)
4. Lấy pixel p có gradient nhỏ nhất ra khỏi queue:
 - Nếu mọi hàng xóm của p cùng nhãn L
→ gán $p \leftarrow L$
 - Nếu hàng xóm có nhiều nhãn khác nhau
→ p là **đường watershed** (nhãn 0)
5. Thêm hàng xóm chưa xử lý của p vào queue
6. Lặp cho đến khi queue rỗng

min1	8	7	90	88	9
	10	5	92	6	11
	85	88	95	89	86
	9	6	91	7	10
min2	8	7	88	9	8

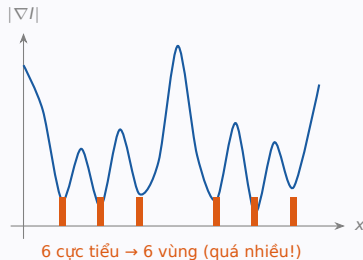
Vấn đề: Over-segmentation

Tại sao xảy ra?

- Ảnh thực có **rất nhiều cực tiểu cục bộ** do nhiễu, kết cấu (texture), gradient nhỏ
- Mỗi cực tiểu $\rightarrow$ một vùng riêng
- Kết quả: hàng trăm/ngày nghìn vùng nhỏ thay vì vài chục vùng có nghĩa

Hệ quả

Watershed “thuần” hầu như không dùng được trực tiếp — luôn cần thêm bước tiền xử lý hoặc marker.



Giải pháp 1: Làm mờ trước khi tính gradient

Áp dụng **Gaussian blur** lên ảnh trước khi tính $|\nabla I|$:

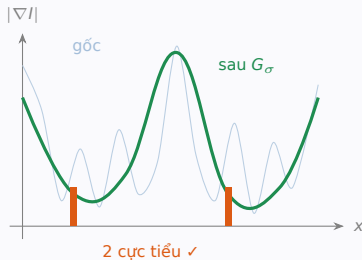
$$|\nabla(G_\sigma * I)|$$

- σ lớn $\rightarrow$ làm mờ nhiều $\rightarrow$ ít cực tiểu hơn
- σ nhỏ $\rightarrow$ ít mờ $\rightarrow$ giữ chi tiết nhưng vẫn nhiều

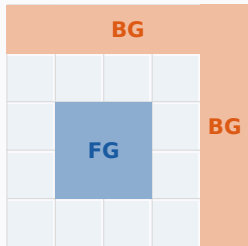
Đánh đổi

- σ lớn: ít vùng, biên không chính xác
- σ nhỏ: nhiều vùng, biên sắc nét

Không giải quyết triệt để — cần cách khác.



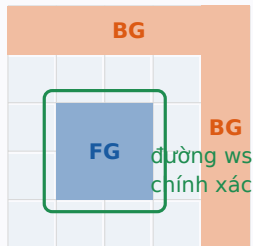
Giải pháp 2: Marker-Controlled Watershed



Hai loại marker:

- **Foreground marker:** pixel bên trong đối tượng quan tâm
- **Background marker:** pixel chắc chắn là nền

Giải pháp 2: Marker-Controlled Watershed



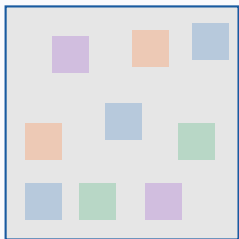
Hai loại marker:

- **Foreground marker:** pixel bên trong đối tượng quan tâm
- **Background marker:** pixel chắc chắn là nền

Kết quả:

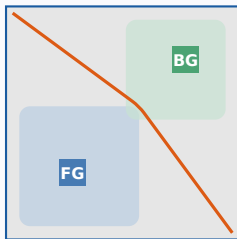
- Chỉ có *đúng số vùng* bằng số marker
- Biên đi dọc theo gradient cao
- Không bị over-segmentation

Marker-Controlled Watershed: So sánh trực quan



Không marker → over-seg.
9 vùng nhỏ

+marker
→



Có marker → 2 vùng ✓
biên dọc gradient cao

Watershed: Độ phức tạp và thực tế

Độ phức tạp:

$$O(N \log N)$$

- N pixel, mỗi pixel vào priority queue một lần
- $O(\log N)$ cho mỗi thao tác queue
- Trong OpenCV:
`cv2.watershed()` chạy rất nhanh

Pipeline thực tế khuyên dùng:

1. Gaussian blur ($\sigma \approx 1-2$)
2. Tính gradient magnitude (Sobel)
3. Tạo marker (thủ công hoặc distance transform)
4. Chạy marker-controlled watershed

Tổng kết Tuần 10: Ba chiến lược phân đoạn vùng

Tiêu chí	Region Growing	Split-and-Merge	Watershed
Cần seed/marker	Seed	Không	Tùy chọn
Hướng xử lý	Bottom-up	Top-down + BU	Flood fill
Thông tin dùng	Cường độ	Phương sai	Gradient
Biên kết quả	Không rõ	Bậc thang	Sắc nét
Over-seg	Không	Không	Có (nếu ko marker)
Phức tạp	$O(N)$	$O(N \log N)$	$O(N \log N)$
Thích hợp nhất	Seed biết trước	Ảnh phân cấp	Ảnh tự nhiên

Kết luận thực hành

Trong hầu hết ứng dụng thực tế: dùng **Marker-Controlled Watershed**
— nhanh, biên chính xác, kiểm soát được số vùng.

Nhìn về phía trước: Tuần 11

Tuần 10 đã làm: phân đoạn dựa trên

- Tiêu chí địa phương: $|I(q) - \mu_R| < \tau$
- Cấu trúc không gian: quad-tree
- Địa hình gradient: flood fill

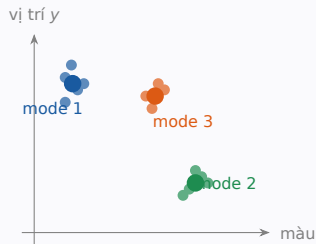
Câu hỏi còn mở:

Nếu ảnh có màu phức tạp, vùng không đồng đều — tiêu chí đồng nhất đơn giản không đủ.

Tuần 11 — Mean-Shift:

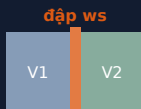
Phân đoạn = tìm *cụm trong không gian đặc trưng*.

Không cần seed, không cần số cụm — tự tìm.



Ý tưởng cốt lõi — Phần 3

Watershed = Flood fill trên địa hình gradient



Biên = gradient cao

Thuần:
over-seg

+marker:
kiểm soát được

Pipeline:

$G_{\sigma} * I$



$|\nabla I|$



marker



watershed

Tuần 11: Mean-Shift — phân đoạn bằng tìm mode trong không gian đặc trưng →

TUẦN 11 · XỬ LÝ ẢNH VÀ THỊ GIÁC MÁY TÍNH

Mean-Shift & Phân tích Mật độ

Density-Based Image Segmentation

KDE · Mean-Shift · Phân đoạn không gian đặc trưng

Câu hỏi mở đầu: Bao nhiêu màu trong ảnh này?

Mỗi pixel là một **điểm trong không gian màu**. Các pixel cùng màu tạo thành **cụm** (cluster).

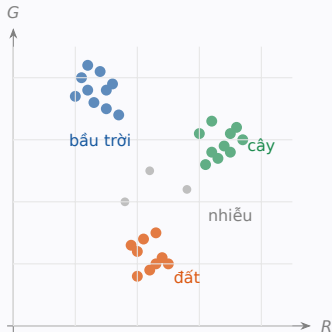
Bài toán phân đoạn = bài toán tìm cụm:

- Tuần 10: dùng *vị trí không gian* + tiêu chí cường độ
- Tuần 11: dùng **mật độ** trong không gian đặc trưng

Câu hỏi cốt lõi

Không biết trước có bao nhiêu cụm.

Làm sao *tự động* tìm ra các cụm?



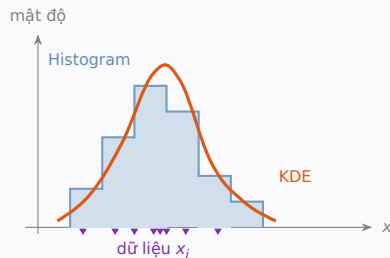
Từ Histogram đến Ước lượng Mật độ Hạt nhân

Histogram — cách ước lượng mật độ quen thuộc:

- Chia trục thành các *bin* có độ rộng h
- Đếm số điểm trong mỗi bin
- Vấn đề: kết quả phụ thuộc vào *vị trí* và *độ rộng* bin
- Không liên tục, không trơn

KDE — phiên bản “mượt mà” của histogram:

- Thay vì đếm theo bin, *đặt một kernel* tại mỗi điểm
- Cộng tất cả kernel lại $\rightarrow$ hàm mật độ *liên tục*



Định nghĩa: Kernel Density Estimation (KDE)

Cho n điểm dữ liệu $x_1, x_2, \dots, x_n$, ước lượng mật độ tại điểm x :

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

Các thành phần:

- $K(\cdot)$: **hàm kernel** — đo “sự đóng góp” của x_i vào mật độ tại x
- h : **bandwidth** — kiểm soát độ rộng ảnh hưởng của mỗi điểm
- $\frac{1}{nh}$: chuẩn hóa để $\int \hat{f}(x) dx = 1$

Kernel phổ biến:

Gaussian : $K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{u^2}{2}}$

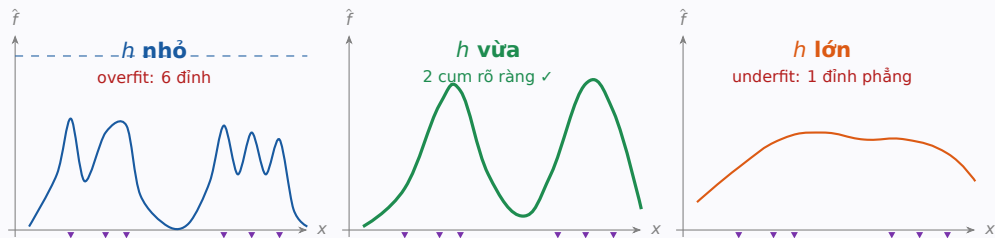
Epanechnikov:

$$K(u) = \frac{3}{4}(1 - u^2) \mathbf{1}_{|u| \leq 1}$$

Gaussian: support vô hạn, mượt.

Epanechnikov: support hữu hạn, tính nhanh hơn.

Bandwidth h : Tham số quan trọng nhất của KDE



Nguyên tắc chọn h

h nhỏ $\rightarrow$ nhạy cảm với nhiễu (overfit) h lớn $\rightarrow$ mất cấu trúc (underfit)

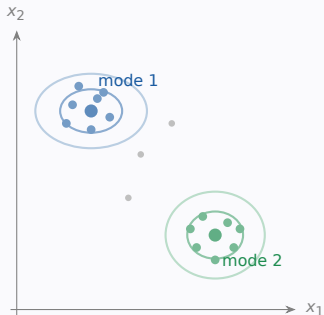
Thực tế: dùng **cross-validation** hoặc **Silverman's rule of thumb**:

$$h = 1.06 \hat{\sigma} n^{-1/5}$$

KDE trong không gian 2D

Mở rộng lên d chiều — tổng quát tự nhiên:

$$\hat{f}(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\|\mathbf{x} - \mathbf{x}_i\|}{h}\right)$$



Đường contour = tập hợp các điểm có cùng mật độ.

Mode (đỉnh cục bộ) = trung tâm của cụm:

- Mỗi mode ứng với một vùng/cụm
- Số mode = số cụm — *tự động xác định*
- Không cần chỉ định trước K

Ý tưởng Mean-Shift

Tìm tất cả mode bằng cách **leo dốc** (gradient ascent) trên hàm $\hat{f}$.

Gradient của KDE và ý tưởng leo dốc

Mục tiêu: tìm điểm $\mathbf{x}^*$ là mode của $\hat{f}$, tức là:

$$\nabla \hat{f}(\mathbf{x}^*) = \mathbf{0} \quad \text{và} \quad \hat{f}(\mathbf{x}^*) \text{ là cực đại cục bộ}$$

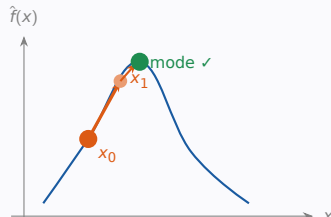
Gradient của KDE Gaussian:

$$\nabla \hat{f}(\mathbf{x}) = \frac{1}{nh^{d+2}} \sum_{i=1}^n (\mathbf{x}_i - \mathbf{x}) K' \left(\frac{\|\mathbf{x} - \mathbf{x}_i\|}{h} \right)$$

Đặt $\nabla \hat{f}(\mathbf{x}) = \mathbf{0}$ và rút gọn:

$$\mathbf{x} = \frac{\sum_i w_i(\mathbf{x}) \mathbf{x}_i}{\sum_i w_i(\mathbf{x})} \quad \text{với } w_i = K' \left(\frac{\|\mathbf{x} - \mathbf{x}_i\|}{h} \right)$$

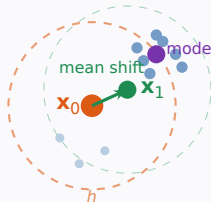
Về phải chính là **trung bình có trọng**



Mean Shift Vector: Trực quan hóa

Tại điểm hiện tại x :

1. Vẽ **cửa sổ** (window) bán kính h xung quanh x
2. Lấy *trung bình có trọng số* của tất cả điểm trong cửa sổ
3. Vị trí trung bình đó là x_{new}
4. Vector từ x đến $x_{\text{new}} = \text{mean shift vector}$
5. Di chuyển đến x_{new} và lặp lại



Tại sao đây là leo dốc?

Trung bình có trọng số luôn dịch về phía *mật độ cao hơn* — mỗi bước tăng $\hat{f}$.

Update Rule của Mean-Shift

Công thức cập nhật

$$\mathbf{x}^{(t+1)} = \frac{\sum_{i=1}^n \mathbf{x}_i \cdot K\left(\frac{\|\mathbf{x}^{(t)} - \mathbf{x}_i\|^2}{h^2}\right)}{\sum_{i=1}^n K\left(\frac{\|\mathbf{x}^{(t)} - \mathbf{x}_i\|^2}{h^2}\right)}$$

Giải thích:

- Tử số: trung bình có trọng số của *tất cả* điểm dữ liệu — điểm gần hơn được trọng số K lớn hơn
- Mẫu số: chuẩn hóa
- Mỗi bước: $\mathbf{x}$ dịch chuyển về trung tâm mật độ cao hơn

Tính chất hội tụ:

- **Hội tụ đảm bảo** với kernel Gaussian và Epanechnikov
- Tốc độ hội tụ: tuyến tính
- Dừng khi $\|\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}\| < \varepsilon$
- Thường hội tụ trong **20-50 bước**

Ví dụ số: 1 bước Mean-Shift

Dữ liệu: 5 điểm trên trục số: $\{1, 2, 8, 9, 10\}$, bandwidth $h = 3$, xuất phát $x^{(0)} = 4$.

Tính trọng số $w_i = \exp\left(-\frac{(x^{(0)} - x_i)^2}{h^2}\right)$:

x_i	$x^{(0)} - x_i$	$(x^{(0)} - x_i)^2/h^2$	$w_i = e^{-(\cdot)}$	$w_i \cdot x_i$
1	+3	$9/9 = 1.00$	0.368	0.368
2	+2	$4/9 = 0.44$	0.644	1.289
8	-4	$16/9 = 1.78$	0.169	1.350
9	-5	$25/9 = 2.78$	0.062	0.556
10	-6	$36/9 = 4.00$	0.018	0.183
Tổng			1.261	3.746

Ý tưởng cốt lõi — Phần 1

KDE = “Histogram mượt mà” với kernel K và bandwidth h

$$\hat{f}(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\|\mathbf{x} - \mathbf{x}_i\|}{h}\right)$$

h nhỏ
overfit

h vừa
✓ cụm rõ ràng

h lớn
underfit

Mode của $\hat{f}$ = trung tâm cụm.

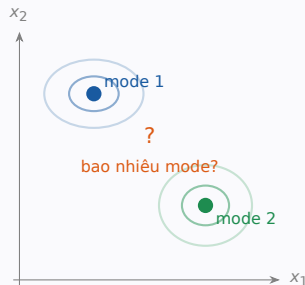
Tiếp theo: Mean Shift — thuật toán lặp để tìm mode.

Đã học: Kernel Density Estimation

- $\hat{f}(\mathbf{x}) = \frac{1}{nh^d} \sum_i K\left(\frac{\|\mathbf{x}-\mathbf{x}_i\|}{h}\right)$
- Bandwidth h kiểm soát độ mịn
- Mode của $\hat{f}$ = trung tâm cụm
- Update rule: $\mathbf{x}^{(t+1)} =$ trung bình có trọng số của lân cận

Vấn đề thực tế:

Làm sao tìm tất cả mode trong ảnh một cách hiệu quả?



Ý tưởng: Chạy Mean-Shift từ mọi điểm

Giải pháp đơn giản nhưng hiệu quả:

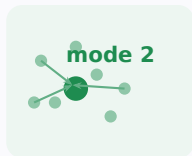
1. Khởi động **một trajectory** (quỹ đạo leo dốc) từ *mỗi điểm dữ liệu*
2. Mỗi trajectory hội tụ về một mode
3. Các điểm cùng hội tụ về một mode → **cùng cụm**



Ý tưởng: Chạy Mean-Shift từ mọi điểm

Giải pháp đơn giản nhưng hiệu quả:

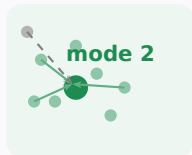
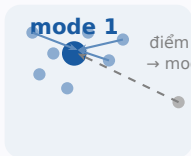
1. Khởi động **một trajectory** (quỹ đạo leo dốc) từ *mỗi điểm dữ liệu*
2. Mỗi trajectory hội tụ về một mode
3. Các điểm cùng hội tụ về một mode → **cùng cụm**



Ý tưởng: Chạy Mean-Shift từ mọi điểm

Giải pháp đơn giản nhưng hiệu quả:

1. Khởi động **một trajectory** (quỹ đạo leo dốc) từ *mỗi điểm dữ liệu*
2. Mỗi trajectory hội tụ về một mode
3. Các điểm cùng hội tụ về một mode → **cùng cụm**



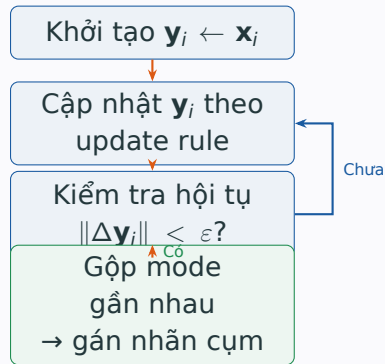
Thuật toán Mean-Shift đầy đủ

Input: tập điểm $\{\mathbf{x}_i\}_{i=1}^n$, bandwidth h , ngưỡng hội tụ ε

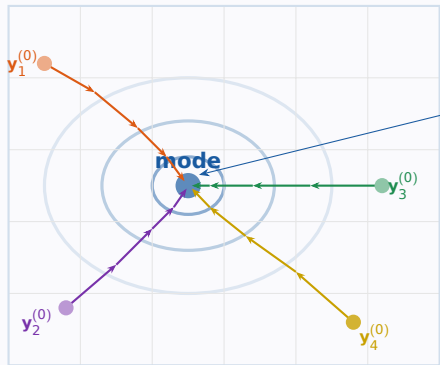
1. Khởi tạo: với mỗi điểm $\mathbf{x}_i$, đặt $\mathbf{y}_i \leftarrow \mathbf{x}_i$
2. Lặp cho đến khi hội tụ:
 - (a) Với mỗi $\mathbf{y}_i$, tính:

$$\mathbf{y}_i \leftarrow \frac{\sum_j K_h(\mathbf{y}_i - \mathbf{x}_j) \mathbf{x}_j}{\sum_j K_h(\mathbf{y}_i - \mathbf{x}_j)}$$

- (b) Nếu $\|\mathbf{y}_i^{\text{new}} - \mathbf{y}_i\| < \varepsilon \rightarrow$ đánh dấu $\mathbf{y}_i$ đã hội tụ
3. Gộp các $\mathbf{y}_i$ nằm trong bán kính ε của nhau
 $\rightarrow$ **cùng mode**
 4. Gán nhãn: $\mathbf{x}_i$ thuộc cụm của mode mà $\mathbf{y}_i$ hội tụ về



Minh hoạ hội tụ: 4 quỹ đạo song song



Tất cả hội tụ
về **một mode**

Tại sao Mean-Shift luôn hội tụ?

Bổ đề cơ bản (Comaniciu & Meer, 2002):

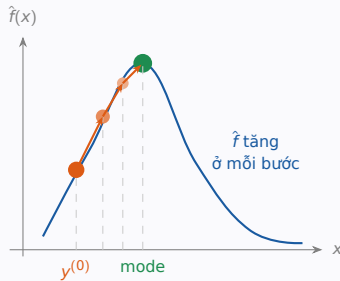
Với kernel K đơn điệu giảm (Gaussian, Epanechnikov):

$$\hat{f}(\mathbf{y}^{(t+1)}) \geq \hat{f}(\mathbf{y}^{(t)})$$

Mỗi bước update **không giảm** mật độ tại điểm hiện tại.

Hệ quả:

- $\hat{f}$ bị chặn trên (có hữu hạn mode)
- Chuỗi $\{\mathbf{y}^{(t)}\}$ đơn điệu tăng theo $\hat{f}$
- → Hội tụ đảm bảo tới cực đại cục bộ



Độ phức tạp tính toán

Chi phí mỗi bước cập nhật:

$O(n)$ cho một trajectory

Phải duyệt qua n điểm để tính trọng số K_h .

Toàn bộ thuật toán:

$$O(n^2 \cdot T)$$

- n : số điểm (pixel)
- T : số bước đến hội tụ (thường 20-50)
- Mỗi trong n trajectory cần $O(n)$ mỗi

Tối ưu hóa trong thực tế:

- **Window cứng**: chỉ tính với điểm trong bán kính h (dùng KD-tree) $\rightarrow O(n \log n \cdot T)$
- **Subsampling**: chạy từ tập con điểm, gán nhãn còn lại theo KNN
- **Pilot bandwidth**: kNN-based adaptive h thay vì h cố định
- Trong OpenCV:
`cv2.pyrMeanShiftFiltering()` —
tối ưu cho ảnh

Cảnh báo

$O(n^2)$ với $n = 512 \times 512 = 262144$ pixel

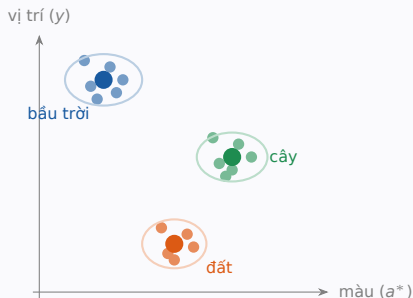
Không gian đặc trưng: Chọn chiều nào?

Vấn đề: pixel trong ảnh có nhiều thông tin — dùng chiều nào?

Không gian	Chiều	Ghi chú
Intensity	1	ảnh xám
RGB	3	nhạy sáng
HSV	3	tốt hơn RGB
CIE L*a*b*	3	tốt nhất
(x,y) + màu	5	spatial-color

Tại sao L*a*b*?

Khoảng cách Euclidean trong L*a*b* tương quan tốt hơn với cảm nhận màu của mắt người.



3 mode rõ ràng → 3 vùng phân đoạn

Không gian joint spatial-color

Comaniciu & Meer (2002) đề xuất dùng không gian **5 chiều**:

$$\mathbf{z}_i = (x_i, y_i, L_i^*, a_i^*, b_i^*)$$

Hai bandwidth riêng biệt:

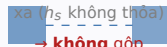
- h_s : **spatial bandwidth** — kiểm soát kích thước vùng trong ảnh (pixel)
- h_r : **range bandwidth** — kiểm soát độ tương đồng màu



gần (h_s thỏa)

nhưng khác màu (h_r không thỏa)

→ **không** gộp



xa (h_s không thỏa)

→ **không** gộp

Metric kết hợp

$$d = \sqrt{\frac{\|\Delta \mathbf{x}_s\|^2}{h_s^2} + \frac{\|\Delta \mathbf{x}_r\|^2}{h_r^2}}$$

Cả hai điều kiện phải thỏa đồng thời

Pixel gần nhau *trong không gian* và

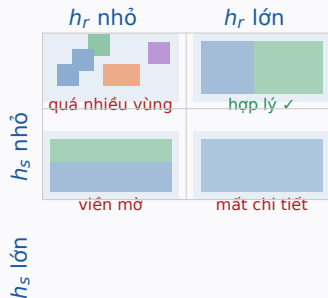
Chọn h_s và h_r trong thực tế

Ảnh hưởng của h_s (spatial):

- Nhỏ: vùng phân đoạn nhỏ, chi tiết, nhiều vùng
- Lớn: vùng lớn, bỏ qua chi tiết nhỏ
- Thường: 7-15 pixel cho ảnh 512×512

Ảnh hưởng của h_r (range/color):

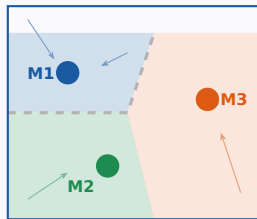
- Nhỏ: chỉ gộp pixel gần giống màu hoàn toàn
- Lớn: gộp cả vùng có màu khác nhau đáng kể
- Thường: 4-20 trong không gian L*a*b*



Basin of Attraction: Mỗi mode thu hút một vùng

Tập hợp tất cả điểm hội tụ về cùng một mode gọi là **basin of attraction** của mode đó.

- Các basin phân chia *toàn bộ* không gian đặc trưng
- Biên giữa hai basin = đường phân đoạn
- Tương tự **Voronoi diagram** nhưng theo cấu trúc mật độ



Không gian đặc trưng

Ảnh xạ về ảnh

Pixel $\mathbf{x}_i$ thuộc vùng phân đoạn k nếu trajectory của nó hội tụ về mode k .

Bước cuối: Gộp các mode gần nhau

Sau khi tất cả trajectory hội tụ, có thể có nhiều *mode giả* do ngưỡng ε và lỗi số học — cần gộp lại:

Thuật toán gộp mode:

1. Tập hợp tất cả điểm hội tụ
 $\mathcal{M} = \{\mathbf{m}_1, \dots, \mathbf{m}_k\}$
2. Với mỗi cặp $(\mathbf{m}_i, \mathbf{m}_j)$:
nếu $\|\mathbf{m}_i - \mathbf{m}_j\| < \delta$
→ hợp nhất thành một mode tại trung điểm
3. Lặp cho đến khi không còn cặp nào gần hơn δ
4. Gán lại nhãn: mỗi điểm dữ liệu thuộc mode tổng hợp gần nhất



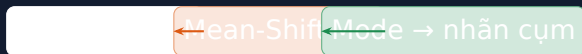
Ý tưởng cốt lõi — Phần 2

Mean-Shift = Gradient ascent trên $\hat{f}$ không cần tính $\nabla \hat{f}$ tường minh

Chạy từ *mỗi* điểm
hội tụ về mode gần nhất

Không cần biết
số cụm K trước

h_s : kích thước vùng
 h_r : độ tương đồng màu



Tiếp theo: áp dụng Mean-Shift vào phân đoạn ảnh thực và so sánh với k-means

$\rightarrow$

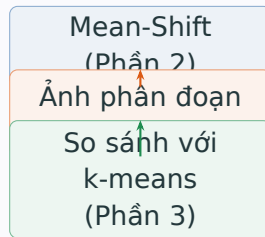
Trong phần trước — Thuật toán Mean-Shift

Đã học: Thuật toán Mean-Shift đầy đủ

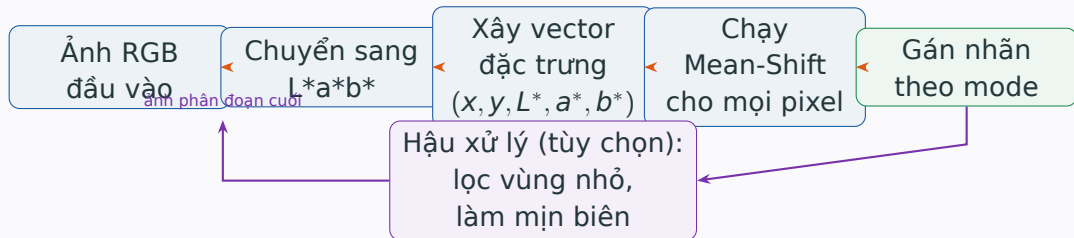
- Chạy trajectory từ mọi điểm $\rightarrow$ hội tụ về mode
- Không gian 5D: (x, y, L^*, a^*, b^*)
- h_s (spatial) + h_r (range) — hai bandwidth
- Gộp mode giả sau hội tụ

Còn thiếu:

- Ảnh xạ kết quả về **ảnh phân đoạn** như thế nào?
- So sánh với **k-means** — khi nào dùng loại nào?
- Đánh giá chất lượng phân đoạn ra



Pipeline Mean-Shift Segmentation đầy đủ



Chuyển RGB → L*a*b*:

RGB → XYZ → L*a*b* (qua công thức D65 illuminant).

Trong OpenCV: `cv2.cvtColor(img, cv2.COLOR_BGR2LAB)`

Hậu xử lý thường dùng:

- Xóa vùng $< N_{min}$ pixel (noise suppression)
- Hợp vùng nhỏ vào vùng lớn gần nhất

Từ mode đến ảnh phân đoạn: Hai cách hiển thị

Cách 1 — Màu trung bình vùng:

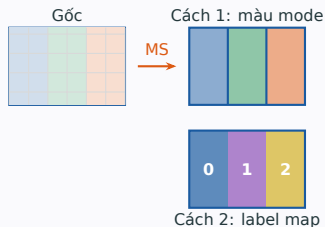
Gán cho mọi pixel trong vùng màu L^*, a^*, b^* của mode.

- Ảnh trông “phẳng” và phân vùng rõ ràng
- Tốt để trực quan hóa kết quả

Cách 2 — Nhãn số (label map):

Mỗi pixel nhận một nhãn số nguyên $0, 1, 2, \dots$

- Dùng cho xử lý tiếp theo (đếm vùng, đo diện tích)
- Tô màu giả (pseudo-color) để hiển thị



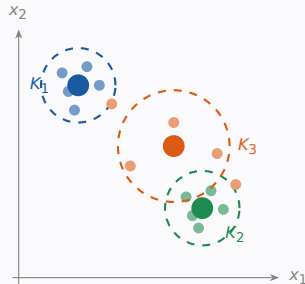
Nhắc lại: k-means clustering

Thuật toán k-means:

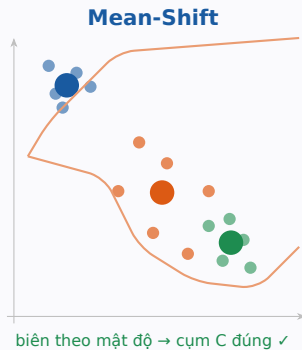
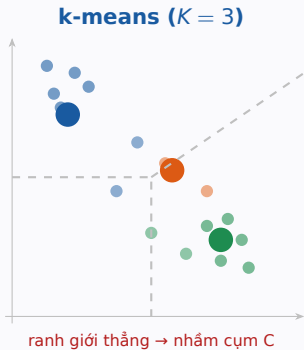
1. Chọn K tâm cụm ban đầu (ngẫu nhiên hoặc k-means++)
2. Gán mỗi điểm về cụm có tâm gần nhất
3. Cập nhật tâm = trung bình điểm trong cụm
4. Lặp 2-3 đến hội tụ

Hàm mục tiêu tối thiểu hóa:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$



So sánh: k-means vs. Mean-Shift — cùng dữ liệu



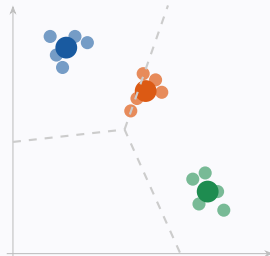
So sánh toàn diện: Mean-Shift vs. k-means

Tiêu chí	k-means	Mean-Shift
Số cụm K	Phải chỉ định	Tự tìm
Hình dạng cụm	Chỉ lồi (convex)	Bất kỳ
Nhạy với khởi tạo	Có	Không
Hàm mục tiêu rõ	Có (J)	Không tường minh
Tốc độ	$O(nKT)$ nhANH	$O(n^2T)$ chẬM
Nhạy với nhiễu	Cao	Trung bình
Tham số cần chỉnh	K	h_s, h_r
Biên cụm	Tuyến tính (Voronoi)	Theo mật độ
Kết quả ổn định	Không	Tốt

Khi nào k-means tốt hơn Mean-Shift?

k-means thắng khi:

- **K biết trước và tin cậy:** phân loại 10 chữ số MNIST $\rightarrow K = 10$
- **Dữ liệu cực lớn:** $n > 10^6$ pixel — k-means $O(nKT)$ nhanh hơn rất nhiều
- **Cụm hình cầu:** dữ liệu tổng hợp từ Gaussian tròn
- **Cần reproducibility:** k-means++ cho kết quả gần tất định

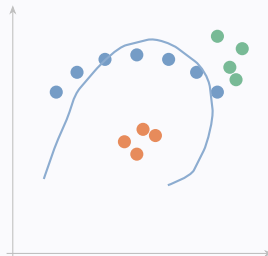


Cụm hình cầu $\rightarrow$ k-means ✓

Khi nào Mean-Shift tốt hơn k-means?

Mean-Shift thắng khi:

- **Không biết K** : ảnh ngoài trời tự nhiên — có bao nhiêu vùng?
- **Cụm hình cong**: các đối tượng kéo dài, vùng cong trong ảnh
- **Cụm không đều kích thước**: đối tượng lớn và nhỏ cùng trong ảnh
- **Cần biên theo cấu trúc**: phân đoạn theo màu tự nhiên thay vì khoảng cách



Cụm hình cong → Mean-Shift ✓

Đánh giá chất lượng phân đoạn

Đánh giá không có ground truth (unsupervised):

Intra-region uniformity — vùng bên trong đồng nhất:

$$U = 1 - \frac{1}{K} \sum_{k=1}^K \sigma_k^2$$

Inter-region contrast — biên giữa vùng rõ:

$$C = \frac{1}{|\mathcal{E}|} \sum_{(i,j) \in \mathcal{E}} \|\mu_i - \mu_j\|$$

Tổng hợp: muốn U cao và C cao.

Đánh giá có ground truth (supervised):

Pixel accuracy:

$$PA = \frac{\text{số pixel gán đúng}}{N}$$

Intersection over Union (IoU) — dùng trong semantic segmentation:

$$IoU_k = \frac{|S_k \cap G_k|}{|S_k \cup G_k|}$$

$$mIoU = \frac{1}{K} \sum_{k=1}^K IoU_k$$

S_k : vùng dự đoán, G_k : ground truth

Hạn chế thực tế của Mean-Shift

1. Tốc độ:

- $O(n^2T)$ — quá chậm cho ảnh HD
- `cv2.pyrMeanShiftFiltering()`
dùng **image pyramid** để tăng tốc
- Thực tế: ảnh 256×256 cần vài giây; ảnh 1920×1080 có thể mất hàng phút

2. Chọn h_s, h_r :

- Không có công thức tự động tốt
- Phụ thuộc nội dung ảnh
- Cần thực nghiệm

3. Ranh giới mờ:

- Vùng chuyển tiếp gradient nhỏ
→ basin của attraction chồng nhau
- Biên không sắc nét bằng Watershed

4. Vùng nhỏ:

- Nhiều mode nhỏ xuất hiện ở vùng kết cấu
- Cần bước hậu xử lý lọc vùng nhỏ

Giải pháp hiện đại:

Dùng **SLIC superpixels** thay Mean-Shift cho tốc

Mean-Shift trong bối cảnh học sâu

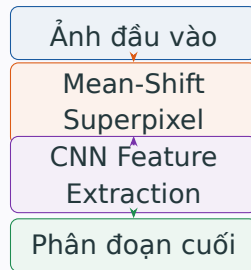
Mean-Shift vẫn còn giá trị dù có deep learning?

1. **Không cần dữ liệu nhãn** — hoàn toàn unsupervised
2. **Hiểu rõ cơ chế** — không phải hộp đen
3. **Tiền xử lý** cho deep pipeline: tạo superpixel, region proposal
4. **Hậu xử lý**: làm mịn biên từ mô hình deep

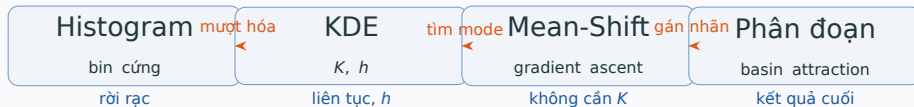
Mean-Shift trong deep learning

Ý tưởng mean-shift được tái sử dụng trong:

- **CamShift**: theo dõi đối tượng (object tracking)
- **DeepMind MHT**: mode-seeking trong latent



Tổng kết Tuần 11: Hành trình từ Histogram đến Mean-Shift



KDE:

- Mật độ = tổng kernel
- Mode = trung tâm cụm
- h kiểm soát độ mịn

Mean-Shift:

- Leo dốc không tường minh
- Hội tụ đảm bảo
- Joint (x, y, L^*, a^*, b^*)

So với k-means:

- Không cần K
- Cụm hình bất kỳ
- Chậm hơn

Nhìn về phía trước: Tuần 12

Tuần 10-11 đã làm: phân đoạn theo *tính chất nội tại* của vùng:

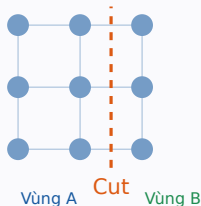
- Tiêu chí đồng nhất (cường độ, màu)
- Mật độ trong không gian đặc trưng
- Địa hình gradient

Còn một góc nhìn khác:

Xem toàn bộ ảnh như một **đồ thị** — pixel là node, độ tương đồng là trọng số cạnh. Phân đoạn = **phân hoạch đồ thị**.

Tuần 12:

Normalized Cut + GrabCut — kết hợp cạnh, đồ thị, và tương tác người dùng.



Ý tưởng cốt lõi — Tuần 11

Phân đoạn = Tìm mode trong không gian đặc trưng

Histogram



KDE

mượt hóa

KDE



Mode

cụm

Mode



Basin

vùng

Không cần K
Cụm bất kỳ
Tất định

$$\text{Mean-Shift: } \mathbf{x}^{(t+1)} = \frac{\sum_i \mathbf{x}_i K_h(\mathbf{x}^{(t)} - \mathbf{x}_i)}{\sum_i K_h(\mathbf{x}^{(t)} - \mathbf{x}_i)}$$

Tuần 12: Graph Cut & GrabCut — phân đoạn theo đồ thị và tương tác người dùng →

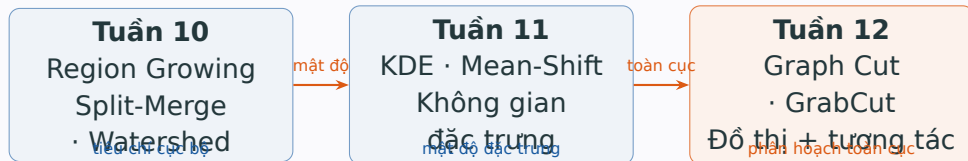
TUẦN 12 · XỬ LÝ ẢNH VÀ THỊ GIÁC MÁY TÍNH

Phân đoạn theo Đồ thị & GrabCut

Graph-Based Segmentation & Interactive Segmentation

Normalized Cut · Min-Cut/Max-Flow · GrabCut · GMM

Hành trình Chương 4: Ba góc nhìn về phân đoạn



Điểm chung tuần 10-11:

- Quyết định dựa trên *tính chất nội tại* mỗi vùng
- Xử lý cục bộ (từng pixel, từng vùng nhỏ)

Tuần 12 — hướng mới:

- Xem ảnh như một **đồ thị**
- Phân đoạn = bài toán **tối ưu toàn cục** trên đồ thị
- Khai thác quan hệ *giữa* mọi cặp vùng cùng lúc

Xây dựng đồ thị từ ảnh

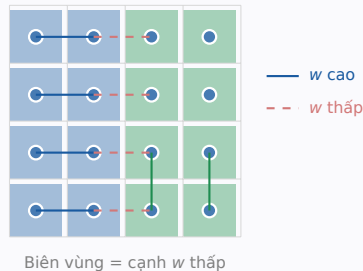
Định nghĩa đồ thị $G = (V, E, w)$:

- V : mỗi pixel (hoặc superpixel) là một **node**
- E : cạnh nối mỗi cặp pixel lân cận
- $w(i, j)$: **trọng số** = độ tương đồng giữa pixel i và j

Hàm trọng số thường dùng:

$$w(i, j) = \exp\left(-\frac{\|I_i - I_j\|^2}{\sigma^2}\right)$$

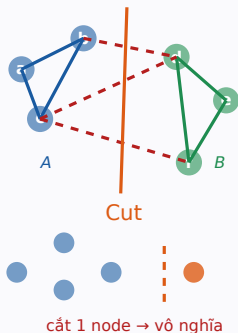
- Pixel giống màu $\rightarrow w$ gần 1
- Pixel khác màu $\rightarrow w$ gần 0



Bài toán Min-Cut: Phân hoạch đồ thị

Mục tiêu: chia V thành hai tập A và $B = V \setminus A$ sao cho tổng trọng số cạnh cắt là nhỏ nhất:

$$\text{cut}(A, B) = \sum_{\substack{u \in A \\ v \in B}} w(u, v)$$



Vấn đề của Min-Cut thuần:

Xu hướng cô lập pixel đơn lẻ

Min-Cut thường chọn cắt ra *một node duy nhất* — vì đó luôn là cách để tổng trọng số cắt nhỏ nhất.

Kết quả vô nghĩa về mặt phân đoạn ảnh.

Normalized Cut: Khắc phục thiên kiến cô lập

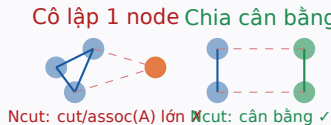
Ý tưởng: chuẩn hóa chi phí cắt theo *tổng độ kết nối* của mỗi phần:

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}$$

với $assoc(A, V) = \sum_{u \in A, v \in V} w(u, v)$ — tổng trọng số cạnh từ A đến toàn bộ đồ thị.

Tại sao hiệu quả?

- Cô lập 1 node nhỏ: $assoc(A, V)$ rất nhỏ $\rightarrow$ Ncut lớn (tệ)
- Phân chia cân bằng: $assoc(A, V) \approx assoc(B, V) \rightarrow$ Ncut nhỏ (tốt)
- Tự động ưu tiên phân đoạn *cân bằng* và *có nghĩa*



Giải Normalized Cut: Bài toán trị riêng

Shi & Malik (2000) chứng minh rằng tối thiểu hóa Ncut tương đương với bài toán **trị riêng tổng quát**:

$$(D - W) \mathbf{y} = \lambda D \mathbf{y}$$

Các ma trận:

- $W \in \mathbb{R}^{n \times n}$: ma trận trọng số
 $W_{ij} = w(i, j)$
- $D \in \mathbb{R}^{n \times n}$: ma trận đường chéo
 $D_{ii} = \sum_j w(i, j)$ (degree)
- $L = D - W$: **Laplacian đồ thị**

Giải pháp thực tế:

1. Tính normalized Laplacian:
 $\tilde{L} = D^{-1/2} L D^{-1/2}$
2. Tìm k vector riêng nhỏ nhất của $\tilde{L}$
3. Dùng k vector riêng làm đặc trưng cho mỗi pixel
4. Chạy k-means trên các đặc trưng này

Spectral Embedding: Biến đổi không gian

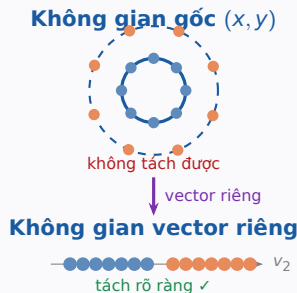
Tại sao cần biến đổi không gian?

Trong không gian pixel (x, y) : hai vùng hình vòng cung *không tách* được bằng đường thẳng.

Sau khi chiếu lên không gian vector riêng: hai vùng *tách biệt rõ ràng* — k-means hoạt động tốt.

Trực quan

Vector riêng của Laplacian “mã hóa” cấu trúc kết nối đồ thị — pixel cùng vùng có giá trị vector riêng gần nhau.



Normalized Cut: Độ phức tạp và giới hạn thực tế

Độ phức tạp:

Xây W : $O(n^2)$ hoặc $O(n \cdot k)$ (sparse)
Tính trị riêng: $O(n^3)$ (tổng quát)
Thực tế: Dùng Lanczos/ARPACK
 $\rightarrow O(n \cdot m)$ với m cạnh

Với ảnh $256 \times 256 = 65536$ pixel:
 $O(n^3)$ là **không khả thi**. Cần
sparse graph (chỉ nối pixel lân cận).

Giới hạn thực tế:

- **Chậm**: ngay cả với sparse graph, tính trị riêng vẫn tốn kém
- **Chỉ phân chia nhị phân**: mỗi lần chạy cho 2 vùng; phân nhiều vùng cần đệ quy
- **Phải chọn k** : số vector riêng = số cụm muốn tìm
- **Biên theo cấu trúc**: không phụ thuộc vào hình dạng cụm
- **Có nền tảng lý thuyết**: bài toán tối ưu rõ ràng

Trong thực tế: dùng **SLIC**

Câu nổi: Từ Normalized Cut đến GrabCut

Ncut tối ưu *không có thông tin người dùng*.

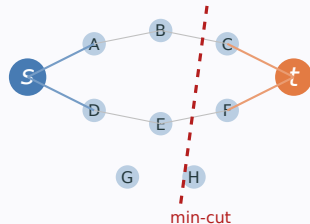
Điều gì xảy ra nếu ta biết một số pixel chắc chắn là foreground hoặc background?

Bài toán mới:

Tìm cắt tối ưu trong đồ thị có **terminal node** (source s và sink t):

- s = foreground (FG)
- t = background (BG)
- Mỗi pixel phải được gán về s hoặc t
- Tối thiểu chi phí cắt

Đây chính là bài toán **min-cut / max-flow** — có thuật toán đa thức hiệu quả.



Định lý Max-Flow Min-Cut

Định lý (Ford-Fulkerson, 1956)

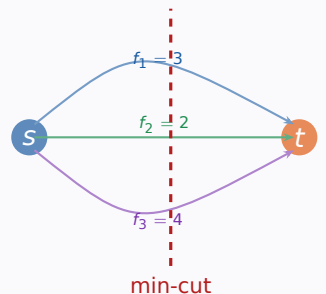
Trong bất kỳ đồ thị có hướng nào với source s và sink t :

$$\text{max-flow}(s \rightarrow t) = \text{min-cut}(s, t)$$

Giá trị luồng cực đại bằng dung lượng cắt cực tiểu.

Ý nghĩa thực tế:

- Thay vì tìm *cắt* nhỏ nhất (khó), ta tìm *luồng* lớn nhất (dễ hơn)
- Tìm max-flow $\Rightarrow$ biết min-cut
- Thuật toán Boykov-Kolmogorov (2004): tối ưu cho đồ thị ảnh, $O(n^3)$ worst case nhưng nhanh trong thực tế



$$\text{max-flow} = 3 + 2 + 4 = 9$$

Xây đồ thị cho bài toán phân đoạn

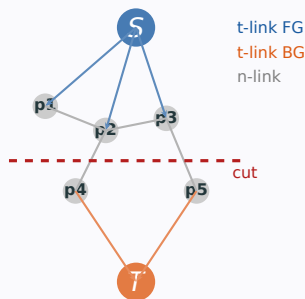
Đồ thị $G = (V, E)$ gồm:

1. Node:

- Mỗi pixel p là một node
- Thêm 2 terminal: source S (FG) và sink T (BG)

2. Cạnh và dung lượng:

- n-link** (p, q) : nối pixel lân cận, $c = w(p, q)$ (độ tương đồng)
- t-link** (p, S) : xác suất pixel p là FG
- t-link** (p, T) : xác suất pixel p là BG



3. Min-cut $\rightarrow$ phân hoạch pixel

Hàm năng lượng của Graph Cut

Bài toán phân đoạn = tối thiểu hóa **hàm năng lượng** E :

$$E(\mathbf{f}) = \underbrace{\sum_{p \in V} D_p(f_p)}_{\text{data term}} + \underbrace{\sum_{(p,q) \in \mathcal{N}} V_{pq}(f_p, f_q)}_{\text{smoothness term}}$$

Data term $D_p(f_p)$:

- Chi phí gán pixel p vào nhãn $f_p \in \{FG, BG\}$
- $D_p(FG) = -\log P(\text{pixel}_p \mid FG \text{ model})$
- $D_p(BG) = -\log P(\text{pixel}_p \mid BG \text{ model})$
- Pixel giống FG $\rightarrow$ chi phí gán FG

Smoothness term $V_{pq}(f_p, f_q)$:

- Chi phí gán hai pixel lân cận vào nhãn *khác nhau*
- $V_{pq}(f_p \neq f_q) = w(p, q)$ (chi phí cao nếu pixel giống nhau)
- $V_{pq}(f_p = f_q) = 0$
- Khuyến khích pixel lân cận giống màu $\rightarrow$ cùng nhãn

Ý tưởng cốt lõi — Phần 1

Ảnh = Đồ thị · Phân đoạn = Min-Cut

Ncut
 $\frac{\text{cut}(A, B)}{\text{assoc}(A, V)}$
tránh cô lập node

Spectral
Laplacian $\tilde{L}$
→ vector riêng
→ k-means

Energy
 $E = D + V$
data term
+ smoothness

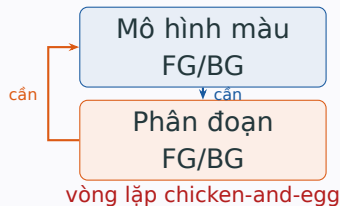
Max-flow = Min-cut → thuật toán đa thức hiệu quả

Tiếp theo: GrabCut — thêm mô hình GMM và tương tác người dùng →

Trong phần trước — Graph-Based Segmentation

Đã học: Normalized Cut + Graph Cut

- Ảnh = đồ thị $G = (V, E, w)$
- Ncut: chuẩn hóa chi phí cắt để tránh cô lập node
- Năng lượng
 $E = D(\text{data}) + V(\text{smoothness})$
- Min-cut = Max-flow → thuật toán đa thức



Vấn đề còn lại:

Làm sao xác định **data term** D_p ?

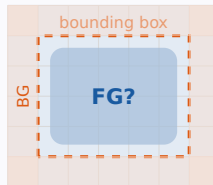
Phải biết mô hình màu của FG/BG —
nhưng ta chưa có!

GrabCut: Giải vòng lặp bằng EM

Rother et al. (2004) — “GrabCut: Interactive Foreground Extraction Using Iterated Graph Cuts”

Ý tưởng giải vòng lặp:

1. Người dùng vẽ **bounding box** quanh đối tượng
2. **Khởi tạo**: pixel ngoài box $\rightarrow$ BG; pixel trong box $\rightarrow$ “có thể FG”
3. **Bước E** (Expectation):
Fit GMM cho FG và BG dựa trên nhãn hiện tại
4. **Bước M** (Maximization):
Chạy Graph Cut với data term từ GMM $\rightarrow$ nhãn mới



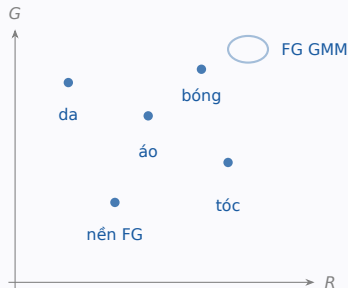
Mô hình màu: Gaussian Mixture Model (GMM)

GrabCut dùng **GMM** với $K = 5$ Gaussian để mô hình hóa phân phối màu:

$$P(\mathbf{c} \mid \text{label}) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(\mathbf{c}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Tại sao GMM thay vì Gaussian đơn?

- FG của ảnh tự nhiên không đồng nhất:
ví dụ người mặc áo có cả màu da, tóc, quần áo
- Mỗi Gaussian con mô hình một “màu đặc trưng” khác nhau
- $K = 5$: đủ linh hoạt, không quá phức tạp



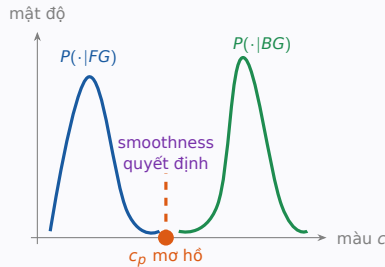
Data term: Từ GMM đến chi phí gán nhãn

Sau khi fit GMM, chi phí gán nhãn cho pixel p có màu $\mathbf{c}_p$:

$$D_p(\text{FG}) = -\log P(\mathbf{c}_p \mid \text{GMM}_{\text{FG}}) \quad ; \quad D_p(\text{BG}) = -\log P(\mathbf{c}_p \mid \text{GMM}_{\text{BG}})$$

Trực quan:

- Nếu màu $\mathbf{c}_p$ giống FG: $P(\mathbf{c}_p \mid \text{FG})$ cao $\rightarrow D_p(\text{FG})$ thấp $\rightarrow$ dễ gán FG
- Nếu màu $\mathbf{c}_p$ giống BG: $D_p(\text{BG})$ thấp $\rightarrow$ dễ gán BG
- Màu mơ hồ: cả hai D đều trung bình $\rightarrow$ smoothness term quyết định

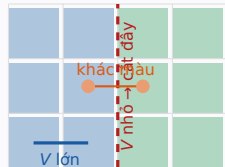


Smoothness term: Biên đi dọc gradient cao

$$V_{pq}(f_p \neq f_q) = \gamma \cdot \exp\left(-\beta \|\mathbf{c}_p - \mathbf{c}_q\|^2\right)$$

Cơ chế:

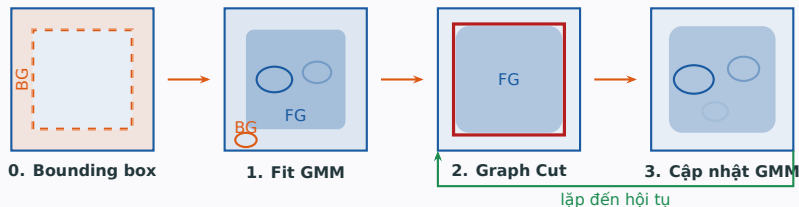
- Hai pixel lân cận *giống màu*: $\|\mathbf{c}_p - \mathbf{c}_q\|$ nhỏ $\rightarrow$ V_{pq} lớn $\rightarrow$ penalty cao nếu cắt $\rightarrow$ khuyến khích cùng nhãn
- Hai pixel *khác màu* (biên thật): V_{pq} nhỏ $\rightarrow$ ít tổn kém nếu cắt ở đây $\rightarrow$ biên đi theo gradient cao



β được tính tự động từ ảnh:

$$\beta = \left(2 \cdot \mathbb{E}\left[\|\mathbf{c}_p - \mathbf{c}_q\|^2\right]\right)^{-1}$$

Pipeline GrabCut: Từng bước chi tiết



Bản chất

GrabCut = Expectation-Maximization (EM) với Graph Cut thay cho bước M.

Mỗi vòng lặp: cả GMM lẫn phân đoạn đều được cải thiện cùng lúc.

Tương tác người dùng: Marker cứng và mềm

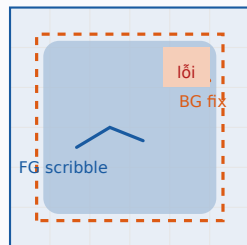
GrabCut hỗ trợ **hai mức độ** tương tác từ người dùng:

Mức 1 — Bounding box (tự động):

- Pixel ngoài box → **hard BG** (t-link BG = ∞)
- Pixel trong box → “unknown” (tính bằng GMM)
- Nhanh, đủ cho nhiều trường hợp

Mức 2 — Scribble (tinh chỉnh):

- Người dùng vẽ thêm **hard FG** hoặc **hard BG** scribble
- Hard pixel → t-link tương ứng = ∞ (không thể cắt)
- Sửa lỗi chỗ GMM chưa phân biệt được



Scribble sửa lỗi GMM

GrabCut hội tụ trong bao nhiêu bước?

Thực nghiệm (Rother et al., 2004):

- Phần lớn ảnh hội tụ sau **5-10 vòng lặp**
- Sau vòng 1: kết quả thô nhưng đã dùng được
- Sau vòng 3: cải thiện rõ ở biên phức tạp
- Sau vòng 5+: ít thay đổi thêm

Điều kiện dừng thực tế:

- Số pixel đổi nhãn $< \epsilon$ (ví dụ 0.1%)
- Hoặc đến số vòng tối đa (thường 5)
- Trong OpenCV: tham số `iterCount`



So sánh: Normalized Cut vs. GrabCut

Tiêu chí	Normalized Cut	GrabCut
Tương tác người dùng	Không	Có (box + scribble)
Mô hình màu	Trọng số cạnh đơn giản	GMM (5 Gaussian)
Số vùng đầu ra	k (chỉ định)	2 (FG/BG)
Thuật toán giải	Trị riêng (chậm)	Max-flow (nhanh)
Chất lượng biên	Trung bình	Rất tốt
Phụ thuộc tham số	σ, k	γ, β (tự động)
Ứng dụng chính	Phân đoạn tự động	Cắt ảnh, xóa nền
Tốc độ (ảnh 512^2)	Chậm ($\sim$ phút)	Nhanh (<1 giây)

Hạn chế của GrabCut và hướng mở rộng

Hạn chế:

- **Chỉ phân đoạn nhị phân**
FG/BG — không dùng cho nhiều đối tượng
- **Biên mờ** ở vùng tóc, lông, thủy tinh — GMM không đủ để mô hình hóa
- **Màu FG $\approx$ BG** — ví dụ người mặc áo trùng màu nền: GMM nhầm
- **Cần bounding box** — không hoàn toàn tự động

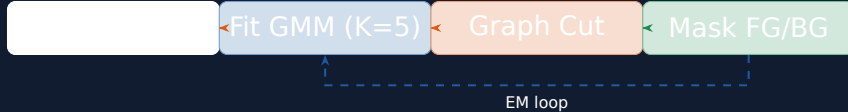
Hướng mở rộng hiện đại:

- **DeepGrabCut**: thay GMM bằng CNN để mô hình hóa FG/BG tốt hơn
- **Matting**: ước lượng alpha matte cho tóc/lông — xử lý biên mờ
- **SAM** (Segment Anything Model, Meta 2023): tương tác bằng prompt (click, box, mask) với ViT → kết quả vượt trội

GrabCut là nền tảng conceptual — hiểu GrabCut là hiểu tính năng SAM

Ý tưởng cốt lõi — Phần 2

GrabCut = GMM + Graph Cut + EM lặp



Data term D từ GMM · Smoothness term V từ gradient màu

Tiếp theo: Tổng kết Chương 4 — so sánh 6 phương pháp, cầu nối sang Ch. 5 →

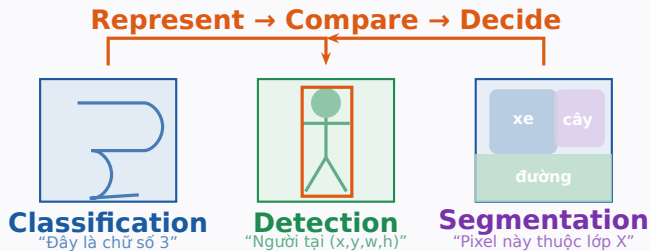
TUẦN 13 · XỬ LÝ ẢNH VÀ THỊ GIÁC MÁY TÍNH

Nhận dạng Ảnh

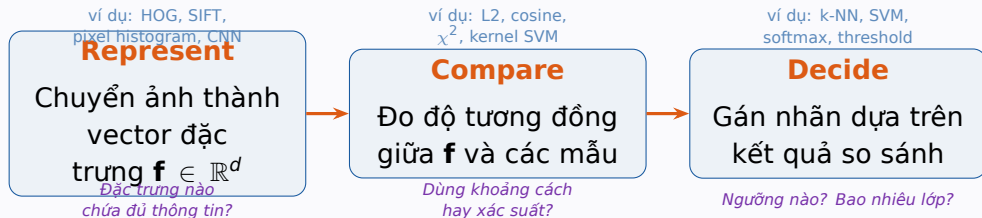
Image Recognition — Chương 5

Pipeline nhận dạng · Không gian đặc trưng · HOG+SVM · CNN · Transfer Learning

Ba bài toán, một cấu trúc chung



Pipeline nhận dạng: Represent → Compare → Decide



Toàn bộ Chương 5 xoay quanh câu hỏi:

Với mỗi bước, thuật toán nào tốt nhất — và tại sao?

Câu trả lời thay đổi theo loại bài toán, kích thước dữ liệu, và tài nguyên tính toán.

Phân biệt ba bài toán: Đầu vào và Đầu ra

Bài toán	Đầu vào	Đầu ra	Độ khó
Classification	Ảnh đã crop xung quanh đối tượng	Nhãn lớp: “mèo”, “chó”, ...	Cơ bản
Detection	Ảnh tùy ý, có thể nhiều đối tượng	Danh sách $(x, y, w, h, \text{lớp})$	Trung bình
Semantic Segmentation	Ảnh tùy ý	Label map: nhãn cho mỗi pixel	Cao
Instance Segmentation	Ảnh tùy ý, nhiều vật thể cùng lớp	Mask riêng cho từng vật thể	Rất cao

Không gian đặc trưng $\mathbb{R}^d$: Ảnh là điểm

Mỗi ảnh được chuyển thành vector

$\mathbf{f} \in \mathbb{R}^d$:

ảnh $\xrightarrow{\phi(\cdot)}$ $\mathbf{f} = (f_1, f_2, \dots, f_d)$

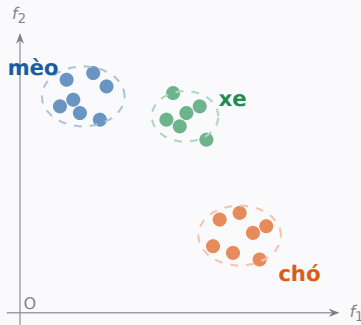
- Ảnh cùng lớp $\rightarrow$ vector **gần nhau** trong $\mathbb{R}^d$
- Ảnh khác lớp $\rightarrow$ vector **xa nhau**
- Nhận dạng = tìm vùng quyết định (decision boundary) trong $\mathbb{R}^d$

Câu hỏi then chốt

Hàm ϕ nào khiến “gần nhau” thực sự có nghĩa?

Đây là toàn bộ bài toán của

Represent.



Không gian đặc trưng $\mathbb{R}^d$: Ảnh là điểm

Mỗi ảnh được chuyển thành vector

$$\mathbf{f} \in \mathbb{R}^d:$$

$$\text{ảnh} \xrightarrow{\phi(\cdot)} \mathbf{f} = (f_1, f_2, \dots, f_d)$$

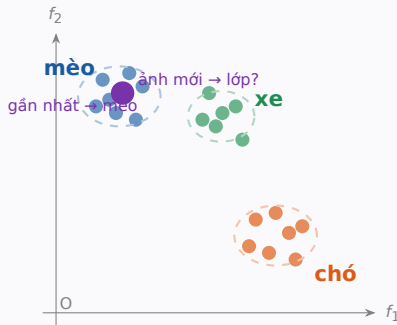
- Ảnh cùng lớp $\rightarrow$ vector **gần nhau** trong $\mathbb{R}^d$
- Ảnh khác lớp $\rightarrow$ vector **xa nhau**
- Nhận dạng = tìm vùng quyết định (decision boundary) trong $\mathbb{R}^d$

Câu hỏi then chốt

Hàm ϕ nào khiến “gần nhau” thực sự có nghĩa?

Đây là toàn bộ bài toán của

Represent.



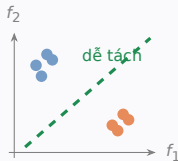
Ba tiêu chí của biểu diễn tốt



$\mathbf{f}$ giống nhau

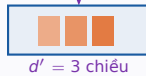
Invariance

Cùng đối tượng, khác điều kiện (scale, rotation, lighting) $\rightarrow$ cùng vector $\mathbf{f}$



Discriminativeness

Lớp khác nhau $\rightarrow$ vector đủ xa nhau để phân loại chính xác



Compactness

Chiều d đủ nhỏ để tính toán hiệu quả; không có chiều thừa

Trade-off

Tăng invariance thường giảm discriminativeness (bỏ quá nhiều thông tin). Tăng discriminativeness thường tăng d (giảm compactness). Thiết kế feature = cân bằng ba tiêu chí này.

So sánh vector: Metric khoảng cách và tương đồng

Bước **Compare** cần đo “khoảng cách” giữa hai vector $\mathbf{f}_a$ và $\mathbf{f}_b$:

Khoảng cách Euclidean (L2):

$$d_2(\mathbf{f}_a, \mathbf{f}_b) = \|\mathbf{f}_a - \mathbf{f}_b\|_2$$

- Dùng khi: vector đặc trưng liên tục, magnitude có nghĩa
- Nhược điểm: nhạy với chiều có giá trị lớn

Cosine similarity:

$$\cos(\mathbf{f}_a, \mathbf{f}_b) = \frac{\mathbf{f}_a \cdot \mathbf{f}_b}{\|\mathbf{f}_a\| \|\mathbf{f}_b\|}$$

Chi-squared distance (χ^2):

$$d_{\chi^2}(\mathbf{f}_a, \mathbf{f}_b) = \sum_i \frac{(f_{a,i} - f_{b,i})^2}{f_{a,i} + f_{b,i}}$$

- Thiết kế riêng cho histogram
- Nhạy với sự khác biệt tương đối, không tuyệt đối
- Dùng khi: histogram màu, histogram HOG, Bag-of-Words

Quy tắc chọn metric

Metric phải phù hợp với *bản chất* của

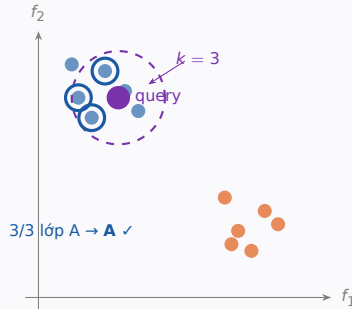
Nearest Neighbor Classifier: Đơn giản nhất có thể

k-Nearest Neighbor (k-NN):

1. Tính khoảng cách từ $\mathbf{f}_{\text{query}}$ đến tất cả n mẫu huấn luyện
2. Lấy k mẫu gần nhất
3. Gán nhãn = nhãn xuất hiện nhiều nhất trong k mẫu đó (majority vote)

Ưu điểm:

- Không cần “train” — chỉ lưu dữ liệu
- Hoạt động tốt khi feature space có cấu trúc rõ
- $k = 1$: Voronoi diagram tự nhiên



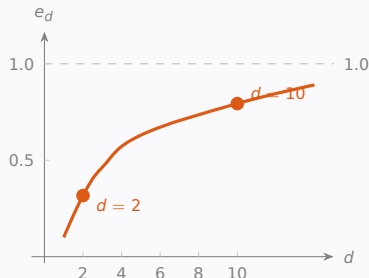
Curse of Dimensionality: Tại sao d lớn là vấn đề

Hiện tượng: Khi d tăng, khoảng cách giữa tất cả các điểm trở nên đồng đều — không còn điểm “gần” và “xa” có nghĩa.

Ví dụ số: Trong hình đơn vị $[0, 1]^d$, để capture 10% thể tích bằng một hypercube, cạnh hypercube phải có độ dài:

$$e_d = (0.1)^{1/d}$$

- $d = 2$: $e = 0.316$ (31% cạnh)
- $d = 10$: $e = 0.794$ (79% cạnh)
- $d = 100$: $e = 0.977$ (98% cạnh!)



Khi $d \rightarrow \infty$, mọi điểm đều “lân cận” nhau.

Giảm chiều: Principal Component Analysis (PCA)

Mục tiêu: chiếu $\mathbf{f} \in \mathbb{R}^d$ xuống $\mathbf{z} \in \mathbb{R}^k$ với $k \ll d$, giữ lại phương sai lớn nhất.

Thuật toán:

1. Trung tâm hóa dữ liệu: $\tilde{\mathbf{f}}_i = \mathbf{f}_i - \bar{\mathbf{f}}$

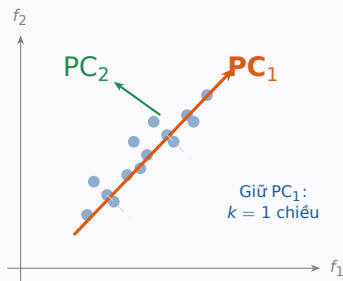
2. Tính ma trận hiệp phương sai:

$$C = \frac{1}{n} \sum_i \tilde{\mathbf{f}}_i \tilde{\mathbf{f}}_i^\top$$

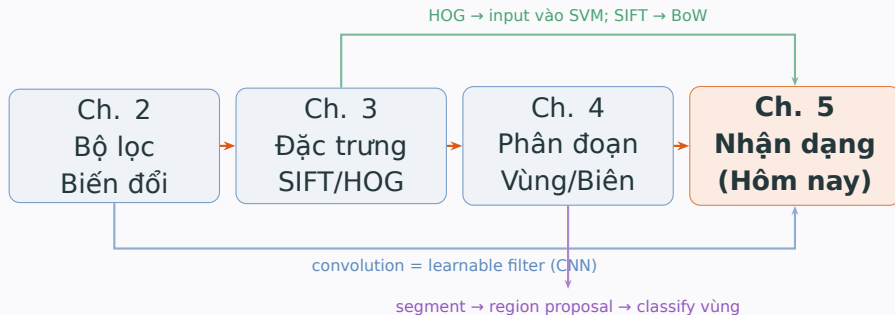
3. Phân tích trị riêng: $C = Q \Lambda Q^\top$

4. Lấy k vector riêng có **trị riêng lớn nhất** làm cơ sở mới

5. Chiếu: $\mathbf{z}_i = Q_k^\top \tilde{\mathbf{f}}_i$



Chương 5 trong bức tranh toàn khóa



Cấu trúc Tuần 13

Phần 1 (hôm nay): Pipeline + không gian đặc trưng — nền tảng lý thuyết

Phần 2: HOG+SVM + Bag-of-Words — hand-crafted feature pipeline

Phần 3: CNN + Transfer Learning — learned feature pipeline

Ý tưởng cốt lõi — Phần 1

Nhận dạng = **Represent** → **Compare** → **Decide**

Biểu diễn tốt:
invariant
discriminative
compact

Ảnh = điểm
trong $\mathbb{R}^d$
cùng lớp
→ gần nhau

d lớn → k-NN thất bại
PCA giảm chiều
giữ phương sai lớn nhất

Metric: L2 cho vector liên tục · cosine cho embedding · χ^2 cho histogram

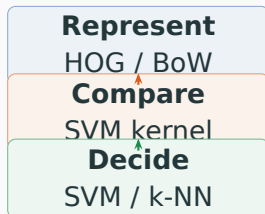
Tiếp theo: HOG+SVM và Bag-of-Words — pipeline hand-crafted feature thực tế

→

Trong phần trước — Pipeline Không gian Đặc trưng

Đã học: Nền tảng nhận dạng

- Pipeline: Represent → Compare → Decide
- Không gian đặc trưng $\mathbb{R}^d$: cùng lớp → gần nhau
- Biểu diễn tốt: invariant, discriminative, compact
- k-NN: đơn giản nhưng chậm; curse of dimensionality
- PCA: giảm chiều giữ phương sai



Câu hỏi Phần này:

Hàm $\phi(\cdot)$ nào — tức là feature nào — thực sự hoạt động tốt trong thực tế

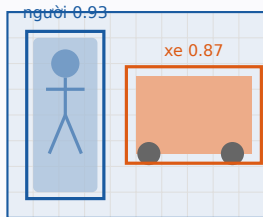
Object Detection: Không chỉ “ảnh này là gì?”

Classification (Phần 1):

- Đầu vào: ảnh đã crop sẵn xung quanh đối tượng
- Đầu ra: một nhãn lớp
- Giả định: đối tượng đã được căn giữa và đúng kích thước

Detection (thực tế hơn):

- Đầu vào: ảnh tùy ý, đối tượng ở vị trí và kích thước bất kỳ
- Đầu ra: danh sách **bounding box** (x, y, w, h , lớp, score)
- Có thể có nhiều đối tượng và nhiều lớp cùng lúc



Sliding Window: Thử mọi vị trí và kích thước

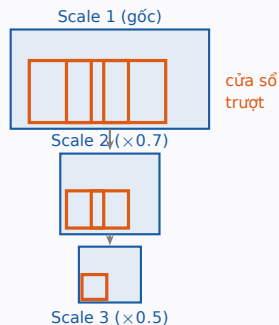
Ý tưởng:

1. Trượt cửa sổ kích thước $W \times H$ qua mọi vị trí (x, y) trong ảnh
2. Tại mỗi vị trí: trích xuất feature, chạy classifier
3. Nếu score > ngưỡng $\rightarrow$ phát hiện đối tượng tại đây
4. Lặp với nhiều kích thước cửa sổ khác nhau (**multi-scale**)

Độ phức tạp:

$$O\left(\frac{W \cdot H}{s^2} \cdot n_{\text{scale}} \cdot T_{\text{clf}}\right)$$

với $s = \text{stride}$, $n_{\text{scale}} = \text{số mức scale}$, $T_{\text{clf}} = \text{thời gian classify}$.



HOG: Histogram of Oriented Gradients

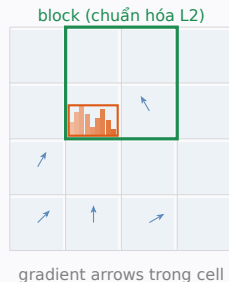
Dalal & Triggs (2005) — feature chuẩn cho người đi bộ. Đã gặp ở Ch. 3 — nay dùng như input cho SVM.

Pipeline tính HOG:

1. Tính gradient (G_x, G_y) — magnitude + orientation
2. Chia ảnh thành **cell** 8×8 pixel
3. Mỗi cell: histogram 9 bin theo orientation (0° – 180°), có trọng số magnitude
4. Nhóm 2×2 cell thành **block**; chuẩn hóa L2 trong block
5. Nối tất cả block $\rightarrow$ vector HOG

Kích thước đặc trưng cho cửa sổ 64×128 :

$$7 \times 15 \text{ blocks} \times 4 \text{ cells} \times 9 \text{ bin} = \mathbf{3780} \text{ chiều}$$



Support Vector Machine: Hyperplane tách lớp tối ưu

Ý tưởng SVM:

Tìm hyperplane $\mathbf{w}^\top \mathbf{f} + b = 0$ sao cho **margin** (khoảng cách đến hai lớp) là lớn nhất:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{f}_i + b) \geq 1$$

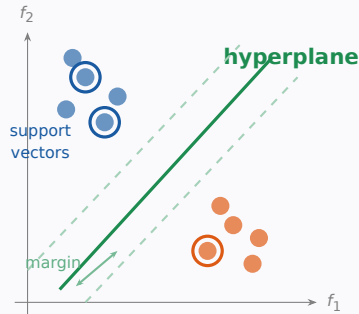
Soft-margin SVM (dữ liệu không tách được tuyến tính):

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i$$

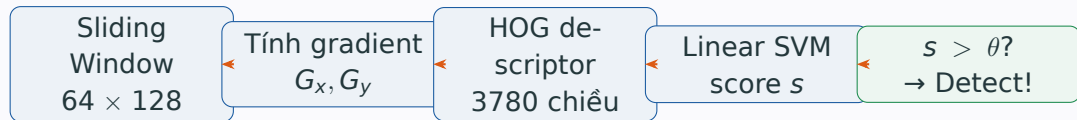
C: trade-off giữa margin lớn và lỗi nhỏ.

Kernel trick:

Thay $\mathbf{f}_i^\top \mathbf{f}_j$ bằng $K(\mathbf{f}_i, \mathbf{f}_j)$ — chiếu lên không gian cao chiều mà không cần tính tường minh.



HOG + SVM: Pipeline Phát hiện Người



Tại sao Linear SVM?

- $T_{\text{clf}} = O(d)$ — rất nhanh, đủ cho sliding window
- Với HOG tốt, linear kernel đã đủ phân biệt người/không-người
- Non-linear kernel (χ^2 -SVM) tốt hơn nhưng chậm hơn $10\times$

Training:

- Positive: ảnh người đã crop và căn chỉnh (~ 2000)
- Negative: vùng ngẫu nhiên không có người (~ 12000)
- Hard negative mining: lấy thêm false positive từ lần chạy đầu làm negative mới

Kết quả (INRIA dataset)

Dalal & Triggs 2005: miss rate 10% ở 1 FPPW — bước ngoặt lớn cho pedestrian detection

Vấn đề: Nhiều bounding box chồng nhau → NMS

Sliding window tạo ra hàng nghìn box,
nhiều box cùng phát hiện *một đối tượng*.

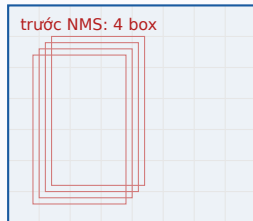
Non-Maximum Suppression (NMS):

1. Sắp xếp tất cả box theo score giảm dần
2. Lấy box có score cao nhất vào kết quả
3. Xóa tất cả box có **IoU** > ngưỡng t với box vừa chọn:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

4. Lặp lại với box tiếp theo còn sót

Thường dùng $t = 0.3-0.5$.



Vấn đề: Nhiều bounding box chồng nhau → NMS

Sliding window tạo ra hàng nghìn box,
nhiều box cùng phát hiện *một đối tượng*.

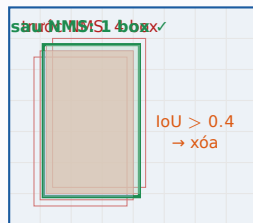
Non-Maximum Suppression (NMS):

1. Sắp xếp tất cả box theo score giảm dần
2. Lấy box có score cao nhất vào kết quả
3. Xóa tất cả box có **IoU** > ngưỡng t với box vừa chọn:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

4. Lặp lại với box tiếp theo còn sót

Thường dùng $t = 0.3-0.5$.



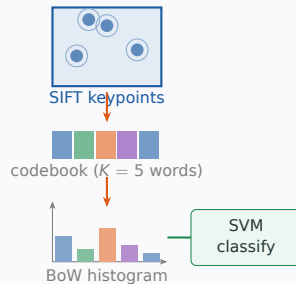
Bag-of-Words: Phân loại ảnh không cần biết vị trí

Ý tưởng — mượn từ NLP:

- Văn bản → đếm tần suất từ → classify
- Ảnh → đếm tần suất “visual word” → classify

Pipeline BoW:

1. Trích SIFT keypoint + descriptor từ nhiều ảnh
2. Chạy k-means trên tất cả descriptor → **codebook** K visual word
3. Mã hóa ảnh: gán mỗi descriptor về visual word gần nhất → histogram K bin
4. Dùng histogram làm feature → SVM/k-NN



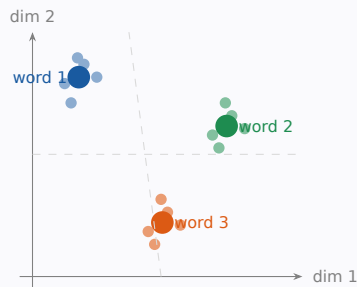
Xây Codebook: k-means trên descriptor

Bước xây codebook (offline):

1. Thu thập ảnh training; trích SIFT từ tất cả ảnh
2. Tập hợp N descriptor 128-chiều ($N \sim 10^6$)
3. Chạy k-means với K cluster (thường $K = 500-4000$)
4. Mỗi centroid = một **visual word**

Bước mã hóa (online, mỗi ảnh mới):

- Trích SIFT → mỗi descriptor → visual word gần nhất (nearest centroid)
- Đếm tần suất → histogram K bin



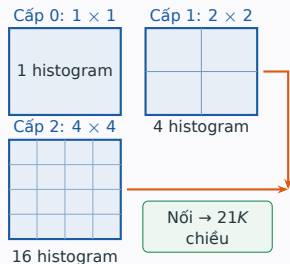
Hạn chế BoW và Spatial Pyramid Matching

Hạn chế của BoW thuần:

- **Mất thông tin không gian:** “chó cắn người” và “người cắn chó” cho cùng histogram
- Descriptor từ khắp ảnh được gộp lại mà không quan tâm vị trí
- Không phân biệt được cấu trúc không gian của đối tượng

Spatial Pyramid Matching (SPM):

- Chia ảnh thành lưới 1×1 , 2×2 , 4×4
- Tính BoW histogram *riêng* cho mỗi ô
- Nối tất cả histogram lại: giữ thông tin “feature này nằm ở đâu”
- Tăng chiều: $K \cdot (1 + 4 + 16) = 21K$



Đánh giá Detection: Precision-Recall & mAP

Định nghĩa IoU cho detection:

Một detection được coi là **True Positive** nếu:

- Nhãn lớp đúng
- $\text{IoU}(\text{pred}, \text{GT}) \geq 0.5$

Precision & Recall:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

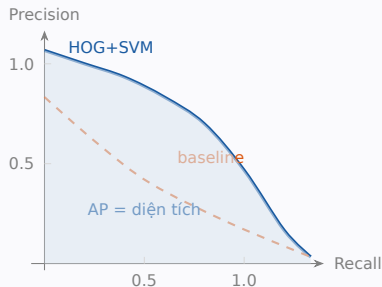
Average Precision (AP):

Diện tích dưới đường P-R curve.

mAP: trung bình AP trên tất cả lớp.

VOC 2007: $\text{IoU}@0.5$ · COCO:

$\text{IoU} \in [0.5:0.95]$



Điểm mạnh và giới hạn của hand-crafted feature

Điểm mạnh:

- **Có thể giải thích:** biết chính xác feature nào được dùng
- **Cần ít dữ liệu:** HOG+SVM train tốt với vài nghìn mẫu
- **Nhanh:** linear SVM predict $O(d)$
- **Ổn định:** kết quả tái lập, không phụ thuộc seed ngẫu nhiên

Giới hạn cơ bản:

- **Feature được thiết kế bằng tay:** không tự thích nghi với bài toán mới
- **HOG nhạy với occlusion, deformation:** người co người đứng thẳng → HOG rất khác nhau
- **BoW mất vị trí:** SPM cải thiện nhưng vẫn thô
- **Pipeline bị chia:** feature và classifier không tối ưu cùng nhau

Câu hỏi dẫn sang Phần 3

Nếu ta học cả feature và classifier cùng lúc từ dữ liệu, kết quả sẽ tốt hơn 157 thế nào?

Ý tưởng cốt lõi — Phần 2

HOG + SVM

Sliding window



HOG (gradient histogram)



Linear SVM + NMS

Bag-of-Words

SIFT descriptor



k-means codebook



Histogram + SVM

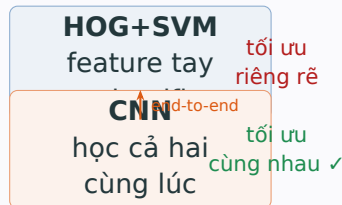
Feature được thiết kế tay → tốt nhưng không tự thích nghi

Tiếp theo: CNN — học feature và classifier cùng lúc từ dữ liệu →

Trong phần trước — HOG+SVM & Bag-of-Words

Đã học: Hand-crafted feature pipeline

- HOG: gradient histogram → 3780 chiều
- Sliding window + linear SVM → detection
- BoW: codebook k-means + SIFT histogram
- NMS loại bỏ box chồng nhau
- Đánh giá: AP, mAP, precision-recall



Giới hạn đã nhận ra:

Feature và classifier tối ưu *riêng rẽ* — không học cùng nhau. CNN giải quyết chính xác điều này.

Tại sao CNN: Learnable Filter thay vì Sobel cố định

Nhắc lại từ Chương 2 — Sobel filter:

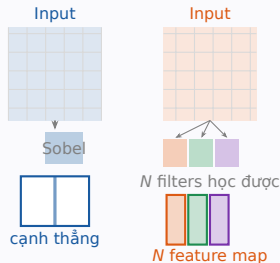
$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad (\text{cố định})$$

Sobel phát hiện cạnh ngang — chỉ một loại đặc trưng, cố định mãi mãi.

CNN Convolutional Layer:

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} \quad (\text{học từ dữ liệu})$$

Mỗi filter học một loại đặc trưng *hữu ích* cho bài toán — cạnh, góc, màu.



Convolutional Layer: Chi tiết kỹ thuật

Đầu vào: tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{in}}$

Tham số: N filter, mỗi filter $\in \mathbb{R}^{k \times k \times C_{in}}$

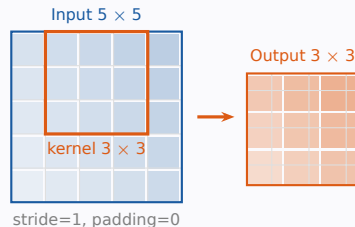
Đầu ra: $\mathbf{Y} \in \mathbb{R}^{H' \times W' \times N}$

Phép tính cho một vị trí (i, j) , filter n :

$$Y_{ij}^{(n)} = \text{bias}_n + \sum_{c=1}^{C_{in}} \sum_{p,q=0}^{k-1} W_{pq}^{(n,c)} \cdot X_{i+p,j+q}^{(c)}$$

Các siêu tham số:

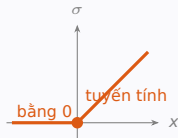
- k : kernel size (thường 3×3 hoặc 1×1)
- s : stride — bước trượt
- p : padding — giữ kích thước output
- $H' = \lfloor (H + 2p - k)/s \rfloor + 1$



Ba bước sau Conv: ReLU · Batch Norm · Pooling

ReLU

$$\sigma(x) = \max(0, x)$$

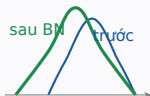


Thêm phi tuyến — không có ReLU, nhiều Conv layer = một Conv layer.

Batch Norm

$$\hat{X} = \frac{X - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

$$y = \gamma \hat{X} + \beta$$



Chuẩn hóa activation theo mini-batch — training ổn định hơn, learning rate cao hơn.

Max Pooling

1	3	2	4
5	6	1	2

 →

6	4
6	2

Giảm kích thước spatial 2×, giữ giá trị lớn nhất trong vùng 2 × 2. Tăng translation invariance.

Receptive Field: Tại sao Stack nhiều Conv Layer?

Receptive field của một node = vùng ảnh đầu vào ảnh hưởng đến giá trị của node đó.

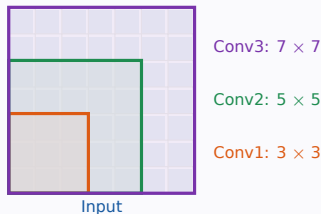
Một Conv 3×3 : receptive field = 3×3

Hai Conv 3×3 : receptive field = 5×5

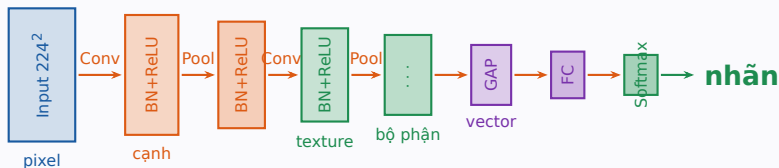
Ba Conv 3×3 : receptive field = 7×7

Tại sao không dùng một Conv 7×7 ?

- 3 lớp 3×3 : tham số
 $= 3 \times 3^2 \times C^2 = 27C^2$
- 1 lớp 7×7 : tham số
 $= 7^2 \times C^2 = 49C^2$
- Stack nhiều lớp nhỏ: **ít tham số hơn, nhiều phi tuyến hơn (ReLU)**



Kiến trúc CNN: Từ Pixel đến Nhãn Lớp



Global Average Pooling (GAP):

Thay vì Flatten → FC lớn, lấy trung bình spatial của mỗi feature map.
Giảm tham số đáng kể; tránh overfitting.

Softmax output:

$$P(\text{lớp}_k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Tổng tất cả lớp = 1 — đây là phân phối xác suất.

Hàm Mất mát và Lan truyền Ngược

Cross-entropy loss — đo sự khác biệt giữa phân phối dự đoán và nhãn thật:

$$\mathcal{L} = - \sum_k y_k \log \hat{y}_k \xrightarrow{\text{one-hot}} \mathcal{L} = - \log \hat{y}_{k^*}$$

với k^* là lớp đúng; $\hat{y}_{k^*}$ là xác suất dự đoán cho lớp đó.

Gradient Descent:

$$W \leftarrow W - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

- η : learning rate — bước cập nhật
- Mini-batch SGD: tính gradient trên batch nhỏ (32-256 ảnh)

Backpropagation — Chain Rule:

$$\frac{\partial \mathcal{L}}{\partial W_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}_L} \cdot \prod_{j=l+1}^L \frac{\partial \mathbf{y}_j}{\partial \mathbf{y}_{j-1}} \cdot \frac{\partial \mathbf{y}_l}{\partial W_l}$$

Gradient lan ngược từ output $\rightarrow$ input qua chain rule.

Mỗi tầng cần lưu activation trong forward pass để dùng trong backward

Transfer Learning: Tại sao hoạt động?

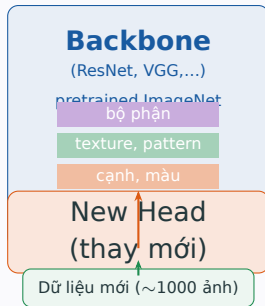
Quan sát thực nghiệm:

CNN được train trên ImageNet (1.2M ảnh, 1000 lớp) học được các feature *tổng quát*:

- Tầng đầu: cạnh, màu, texture (giống Gabor, HOG)
- Tầng giữa: góc, kết cấu, pattern lặp
- Tầng sâu: bộ phận đối tượng (mắt, bánh xe, ...)

Feature ở tầng đầu và giữa *hữu ích cho hầu hết bài toán*, không chỉ ImageNet.

→ Dùng lại weight pretrained thay vì train từ đầu.



Hai chế độ: Feature Extraction vs. Fine-tuning

Chế độ 1 — Feature Extraction:

1. *Đóng băng* toàn bộ backbone (không cập nhật weight)
2. Chỉ train *head mới* (FC + Softmax)
3. Backbone = fixed feature extractor

Khi nào dùng:

- Dữ liệu rất ít (< 1000 ảnh)
- Dữ liệu mới giống với ImageNet
- Tài nguyên GPU hạn chế
- Train nhanh — chỉ vài phút

Chế độ 2 — Fine-tuning:

1. Load pretrained weight
2. Thay head mới
3. Train *toàn bộ mô hình* với learning rate *nhỏ* (10^{-4} – 10^{-5})
4. Tầng đầu: learning rate rất nhỏ; tầng sau: lớn hơn

Khi nào dùng:

- Dữ liệu đủ nhiều (> 5000 ảnh)
- Dữ liệu khác nhiều so với ImageNet (ảnh y tế, ảnh vệ tinh)
- Cần độ chính xác cao nhất

Transfer Learning trong thực tế: PyTorch

Pipeline cụ thể (PyTorch):

1. Load pretrained model:

```
model = resnet50(pretrained=True)
```

2. Đóng băng backbone:

```
for p in model.parameters():  
    p.requires_grad = False
```

3. Thay head:

```
model.fc = nn.Linear(2048, n_classes)
```

4. Optimizer chỉ update head:

```
optimizer = Adam(model.fc.parameters(),  
                  lr=1e-3)
```

5. Train loop: 10-30 epoch thường đủ.

Train từ đầu vs. Transfer

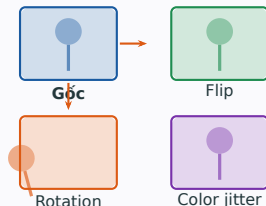
Từ đầu	Transfer
Dữ liệu Thường	~1K-10K phút-giờ
GPU cần	1 GPU đủ tốt ngay
Overfit	ít hơn

Data Augmentation: Tăng hiệu quả khi ít dữ liệu

Ý tưởng: tạo ra nhiều biến thể của ảnh gốc — không tốn công thu thập thêm dữ liệu.

Các phép biến đổi phổ biến:

- **Geometric:** flip ngang/dọc, random crop, rotation ($\pm 30^\circ$), perspective
- **Color:** brightness, contrast, saturation jitter; grayscale ngẫu nhiên
- **Noise:** Gaussian noise, blur, JPEG artifact
- **Advanced:** CutOut, MixUp, CutMix, RandAugment



Kết nối với Chương 2

Geometric transforms (Ch. 2): homography, affine — chính là những phép biến đổi tương tự. Data

Đánh giá CNN: Confusion Matrix & Per-class Metric

Confusion Matrix — $K \times K$:

C_{ij} = số ảnh thuộc lớp i được dự đoán là lớp j .

Thật

Mèo

Chó

Xe

	Mèo	Chó	Xe
Mèo	45	3	2
Chó	4	38	8
Xe	1	6	41

Khi nào cần confusion matrix?

Khi accuracy tổng thể cao nhưng muốn biết lớp nào hay bị nhầm với lớp nào — để debug và cải thiện.

Per-class metrics:

Ví dụ lớp “Mèo” (hàng 3):

$$\text{Precision} = \frac{45}{45 + 4 + 1} = 90\%$$

$$\text{Recall} = \frac{45}{45 + 3 + 2} = 90\%$$

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Overall accuracy:

$$\text{Acc} = \frac{45 + 38 + 41}{45 + 38 + 41 + \text{errors}}$$

Ý tưởng cốt lõi — Chương 5

CNN

Learnable filter
Conv → BN → ReLU → Pool
Receptive field tăng dần
Backprop + Cross-entropy

Transfer Learning

Pretrained weight
= feature tổng quát
Feature Extraction:
đóng băng backbone
Fine-tuning:
LR nhỏ, train toàn bộ

Toàn khóa

Ch.2: pixel → filter
Ch.3: filter → feature
Ch.4: feature → region
Ch.5: region → label

Represent → Compare → Decide

Từ Sobel cố định đến filter học được —
cùng một phép tính tích chập, khác ở chỗ *ai thiết kế*